

1.7 Esercizi svolti

Problema 1.1: Linguaggio rovesciato

Dato un linguaggio L e il suo linguaggio rovesciato $L^R = \{w^R \mid w \in L\}$, dimostrare che

$$L \in \text{REG} \implies L^R \in \text{REG}$$

Dimostrazione.

- Dato $L \in \text{REG}$, sia $D = (Q, \Sigma, \delta, q_0, F)$ il DFA tale che $L = L(D)$
- Definiamo quindi un primo NFA $N = (Q', \Sigma, \delta', q_0, \{q_f\})$ tale che:
 - q_f è il nuovo unico stato accettante aggiunto
 - $Q' = Q \cup \{q_f\}$
 - $\forall q \in Q, a \in \Sigma \quad \delta'(q, a) = \delta(q, a)$, ossia tutti gli archi rimangono invariati
 - $\forall q \in F \quad \delta'(q, \varepsilon) = q_f$, ossia tutti gli stati finali precedenti hanno un ε -arco verso q_f
- A questo punto, per costruzione stessa di N ne segue che:

$$w \in L = L(D) \iff w \in L(N)$$

dunque che $L = L(D) = L(N)$

- Definiamo quindi un secondo NFA $N^R = (Q', \Sigma, \delta'', q_f, \{q_0\})$ avente tutti gli archi invertiti rispetto ad N , ossia tale che:

$$\forall q \in Q, a \in \Sigma_\varepsilon \quad \delta''(q, a) = R_q$$

dove $R_q = \{p \in Q' \mid \delta'(p, a) = q\}$

- A questo punto, per costruzione stessa di N' ne segue che:

$$w \in L = L(N) \iff w^R \in L(N^R)$$

dunque che $L^R = L(N)^R = L(N^R) \in \text{REG}$

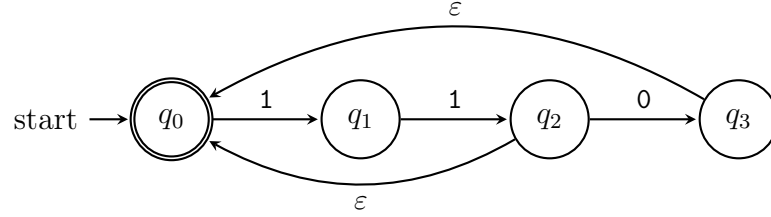
□

Problema 1.2

Dato il linguaggio $L = \{11, 110\}^*$, costruire un NFA N con 4 stati che riconosca L . Convertire il NFA in un DFA M equivalente.

Soluzione:

- Definiamo il seguente NFA $N = (Q_N, \Sigma, \delta_N, q_0^N, F_N)$ come:



dove risulta evidente che $L(N) = L$

- Convertiamo quindi N nel DFA equivalente $D = (Q_D, \Sigma, \delta_D, q_0^D, F_D)$, dove:
 - $Q_D = \mathcal{P}(Q_N) = \mathcal{P}(\{q_1, q_2, q_3, q_4\})$
 - Dato $R \in Q_D$, sia:

$$E(R) = \{q \in Q_N \mid \exists p \in R \text{ che raggiunge } q \text{ in } N \text{ tramite solo } \varepsilon\text{-archi}\}$$

- $q_0^D = E(\{q_0^N\}) = \{q_0^N\}$
- $F_D = \{R \in Q_D \mid R \cap F_N \neq \emptyset\}$
- Dati $a \in \Sigma$ e $R \in Q_D$ vale che:

$$\delta_D(R, a) = \bigcup_{r \in R} E(\delta_N(r, a))$$

- Per costruzione di D , si ha che:

$$w \in L(D) \iff \delta_D^*(q_0^D, w) \in F_D \iff$$

$$\exists r \in \delta_D^*(q_0^D, w), \delta_N^*(q_0^N) = r \in F_N \iff w \in L(N)$$

concludendo che $L(N) = L(D)$

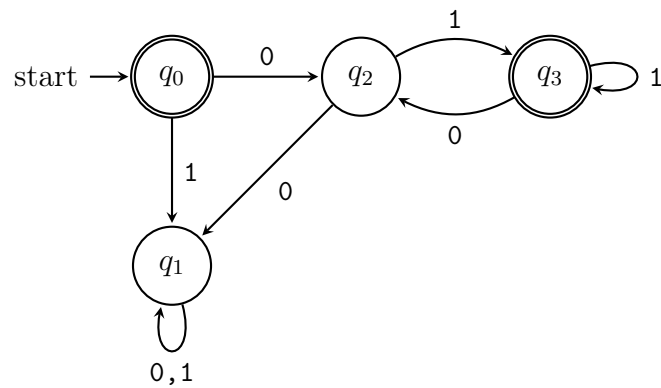
Problema 1.3: Complemento di un'espressione regolare

Data l'espressione regolare $R = (01^+)^*$, costruire il DFA D tale che:

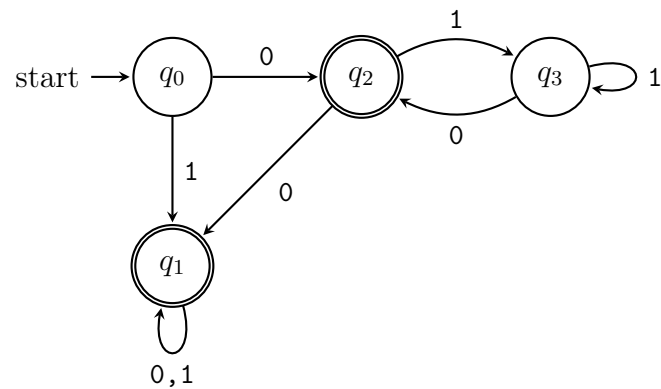
$$L(D) = \{w \in \{0, 1\}^* \mid w \notin L(R)\}$$

Soluzione:

- Prima di tutto, costruiamo un DFA D_R tale che $L(D_R) = L(R)$:



- A questo punto, ci basta costruire il DFA D tale che $L(D) = \overline{L(D_R)}$ utilizzando la [Chiusura del complemento in REG](#):



Problema 1.4

Dato il linguaggio $L = \{w \in \{0,1\}^* \mid |w|_0 = |w|_1\}$, dimostrare che $L \notin \text{REG}$

Dimostrazione.

- Supponiamo per assurdo che L sia regolare, implicando che per esso debba valere il pumping lemma, dove p è la lunghezza del pumping
- Consideriamo quindi la stringa $w := 0^p 1^p \in L$. Poiché $|w| \geq p$, possiamo suddividerla in tre sottostringhe $x, y, z \in \Sigma^*$ tali che $w = xyz$
- Poiché la terza condizione del pumping lemma impone che $|xy| \leq p$ e poiché $w := 0^p 1^p$, ne segue che $xy = 0^m$ e $z = 0^{p-m} 1^p$, dove $m \in [1, p]$
- Inoltre, per la seconda condizione, si ha che $|y| > 0$, dunque necessariamente si ha che $x = 0^{m-k}$ e $y = 0^k$, dove $k \in [1, m]$
- A questo punto, consideriamo la stringa $xy^0 z$. Notiamo immediatamente che

$$xy^0 z = 0^{m-k} (0^k)^0 0^{p-m} 1^p = 0^{m-k} 0^{p-m} 1^p = 0^{p-k} 1^p$$

$$\implies |xy^0 z|_0 \neq |xy^0 z|_1 \implies xy^0 z \notin L$$

contraddicendo la prima condizione del lemma per cui si ha che $\forall i \in \mathbb{N} \quad xy^i z \in L$

- Dunque, ne segue necessariamente che L non sia regolare

□

Problema 1.5

Dato il linguaggio $L = \{1^{n^2} \mid n \in \mathbb{N}\}$, dimostrare che $L \notin \text{REG}$

Dimostrazione.

- Supponiamo per assurdo che L sia regolare, implicando che per esso debba valere il pumping lemma, dove p è la lunghezza del pumping
- Consideriamo quindi la stringa $w := 1^{p^2} \in L$. Poiché $|w| \geq p$, possiamo suddividerla in tre sottostringhe $x, y, z \in \Sigma^*$ tali che $w = xyz$
- Poiché la terza condizione del lemma impone che $|xy| \leq p$ e poiché $w := 1^{p^2}$, ne segue che $xy = 1^m$ e $z = 1^{p^2-m}$, dove $m \in [1, p]$
- Inoltre, per la seconda condizione del lemma, si ha che $|y| > 0$, dunque necessariamente si ha che $x = 1^{m-k}$ e $y = 1^k$, dove $k \in [1, m]$
- A questo punto, consideriamo la stringa $xy^0 z$. Notiamo immediatamente che

$$xy^0 z = 1^{m-k} (1^k)^0 1^{p^2-m} = 1^{p^2-k}$$

- Tuttavia, poiché $k \in [1, p]$, ne segue che $\nexists n \in \mathbb{N} \mid n^2 = p^2 - k$, implicando dunque che $xy^0z \notin L$, contraddicendo la prima condizione del lemma per cui si ha che $\forall i \in \mathbb{N} \quad xy^iz \in L$
- Dunque, ne segue necessariamente che L non sia regolare

□

Problema 1.6

Dato il linguaggio $L = \{1^i \# 1^j \# 1^{i+j} \mid i, j \in \mathbb{N}\}$, dimostrare che $L \notin \text{REG}$

Dimostrazione.

- Supponiamo per assurdo che L sia regolare, implicando che per esso debba valere il pumping lemma, dove p è la lunghezza del pumping
- Consideriamo quindi la stringa $w := 1^p \# 1^p \# 1^{2p} \in L$. Poiché $|w| \geq p$, possiamo suddividerla in tre sottostringhe $x, y, z \in \Sigma^*$ tali che $w = xyz$
- Poiché la terza condizione del lemma impone che $|xy| \leq p$ e poiché $w := 1^p \# 1^p \# 1^{2p}$, ne segue che $xy = 1^m$ e $z = 1^{p-m} \# 1^p \# 1^{2p}$, dove $m \in [1, p]$
- Inoltre, per la seconda condizione del lemma, si ha che $|y| > 0$, dunque necessariamente si ha che $x = 1^{m-k}$ e $y = 1^k$, dove $k \in [1, m]$
- A questo punto, consideriamo la stringa xy^0z . Notiamo immediatamente che:

$$xy^0z = 1^{m-k}(1^k)^0 1^{p-m} \# 1^p \# 1^{2p} = 1^{p-k} \# 1^p \# 1^{2p} \implies xy^0z \notin L$$

contraddicendo la prima condizione del lemma per cui si ha che $\forall i \in \mathbb{N} \quad xy^iz \in L$

- Dunque, ne segue necessariamente che L non sia regolare

□

Problema 1.7

Dato il linguaggio $L = \{c^4 a^n b^m \mid n, m \in \mathbb{N}, n \geq m\}$, dimostrare che $L \notin \text{REG}$

Dimostrazione.

- Supponiamo per assurdo che L sia regolare, implicando che per esso debba valere il pumping lemma, dove p è la lunghezza del pumping
- Consideriamo quindi la stringa $w := c^4 a^p b^p \in L$. Poiché $|w| \geq p$, possiamo suddividerla in tre sottostringhe $x, y, z \in \Sigma^*$ tali che $w = xyz$
- Poiché la terza condizione del lemma impone che $|xy| \leq p$ e poiché $w := c^4 a^p b^p$, ne segue che $xy = c^4 a^k$ e $z = a^{p-k} b^p$, dove $k \in [0, p-4]$

- Per la seconda condizione del lemma, si ha che $|y| > 0$, dunque y necessariamente $\exists u \in \Sigma^*$ tale che $y = ua$, ossia y contiene almeno un simbolo a . Abbiamo quindi quattro sotto-casi:

1. Se $x = c$ e $y = c^3a^k$ allora notiamo che $xy^0z = ca^{p-k}b^p \notin L$
2. Se $x = c^2$ e $y = c^2a^k$ allora notiamo che $xy^0z = c^2a^{p-k}b^p \notin L$
3. Se $x = c^3$ e $y = ca^k$ allora notiamo che $xy^0z = c^3a^{p-k}b^p \notin L$
4. Se $x = c^4a^h$ e $y = a^{k-h}$, dove $h \in [0, k-1]$ allora notiamo che $xy^0z = c^4a^ha^{p-k}b^p = c^4a^{p-k+h}b^p \notin L$

In altre parole, in ogni caso possibile viene contraddetta la prima condizione del lemma per cui si ha che $\forall i \in \mathbb{N} \ xy^iz \in L$

- Dunque, ne segue necessariamente che L non sia regolare

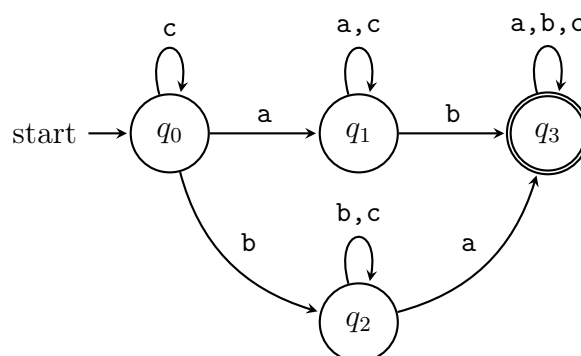
□

Problema 1.8

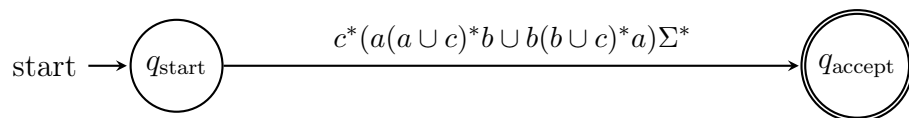
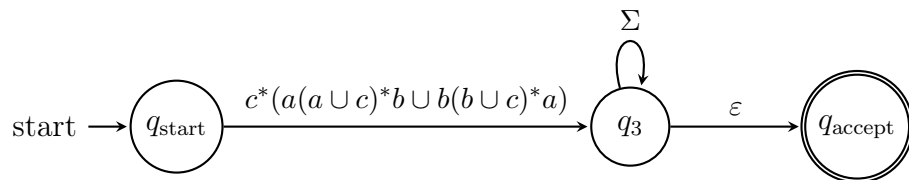
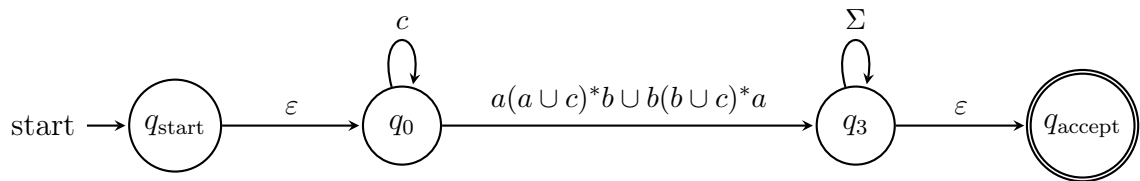
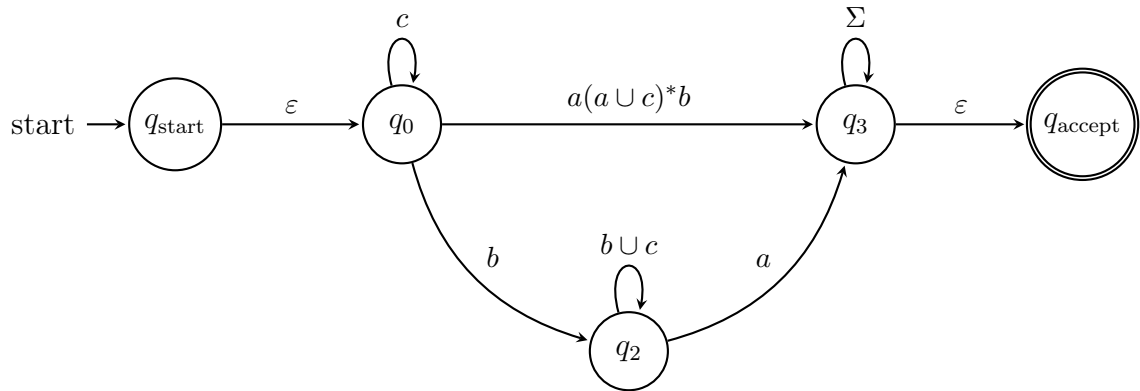
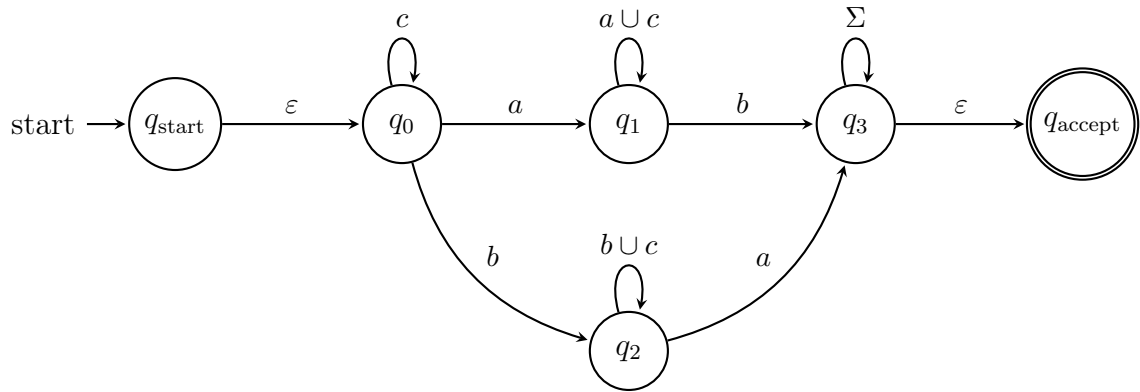
Sia $\Sigma = \{a, b, c\}$. Determinare un'espressione regolare $R \in \text{re}(\Sigma)$ descrivente il linguaggio di Σ composto dalle stringhe contenenti almeno una a ed almeno una b . Determinare inoltre un DFA D che riconosca lo stesso linguaggio.

Soluzione:

- Nonostante il problema inviti alla determinazione dell'espressione regolare e poi del DFA ad essa equivalente, trovare quest'ultimo risulta molto più rapido



- A questo punto, osservando il DFA possiamo già notare che l'espressione regolare ad esso equivalente corrisponda a $c^*(a(a \cup c)^*b \cup b(b \cup c)^*a)\Sigma^*$
- Volendo procedere più rigorosamente, possiamo ricavare tale espressione regolare convertendo il DFA costruito nel suo GNFA equivalente, per poi ridurre al minimo tale GNFA, ottenendo l'espressione regolare
- Definiamo quindi il GNFA equivalente (del quale vengono omesse le sue transizioni etichettate con $\emptyset$), per poi procedere con la sua riduzione:



Problema 1.9

Sia $A = (Q, \Sigma, \delta, q_0, \{q_f\})$ un DFA e si supponga che $\forall a \in \Sigma$ si abbia che $\delta(q_0, a) = \delta(q_f, a)$. Si dimostri che:

1. Per ogni $w \neq \varepsilon$, si ha che $\delta^*(q_0, w) = \delta^*(q_f, w)$, dove δ^* è la funzione di transizione estesa
2. Si mostri che se $x \neq \varepsilon \in L(A)$, allora $\forall k > 0 \in \mathbb{N} \ x^k \in L(A)$

Dimostrazione.

1. • Data $w := ay \in \Sigma^*$, dove $a \in \Sigma, y \in \Sigma^*$, si ha che:

$$\delta^*(q_0, w) = \delta^*(q_0, ay) = \delta^*(\delta(q_0, a), y) = \delta^*(\delta(q_f, a), y) = \delta^*(q_f, ay) = \delta^*(q_f, w)$$

2. • Data $x \neq \varepsilon \in L(A)$, per definizione stessa si ha che:

$$x \neq \varepsilon \in L(A) \iff \delta^*(q_0, x) \in \{q_f\} \iff \delta^*(q_0, x) = q_f$$

- Procediamo quindi per induzione su $k \in \mathbb{N}$

Caso base ($k = 1$):

$$- x \neq \varepsilon \in L(A) \implies x^1 \in L(A)$$

Ipotesi induttiva:

- Per $k > 1 \in \mathbb{N}$, si ha che:

$$x \neq \varepsilon \in L(A) \implies x^k \in L(A)$$

Passo induttivo:

- Per $k + 1 \in \mathbb{N}$, notiamo che:

$$\delta^*(q_0, x^{k+1}) = \delta^*(q_0, xx^k) = \delta^*(\delta^*(q_0, x), x^k) = \delta^*(q_f, x^k)$$

a questo punto, tramite il punto 1. si ha che:

$$\delta^*(q_f, x^k) = \delta^*(q_0, x^k) = q_f \implies \delta^*(q_0, x^{k+1}) \in \{q_f\} \iff x^{k+1} \in L(A)$$

□

Problema 1.10: Somma di cifre come multiplo di k

Sia L_k il linguaggio di tutte le stringhe su alfabeto $\Sigma = \{0, 1, \dots, 9\}$ la cui somma delle cifre è un multiplo di k , dove $k \in \mathbb{N}$. Descrivere formalmente un DFA che riconosca L_k per ogni $k \in \mathbb{N}$. Provare la correttezza del DFA proposto e disegnare il diagramma di transizione degli stati del DFA per il caso $k = 4$

Dimostrazione.

- Dato $k \in \mathbb{N}$, sia L_k il linguaggio richiesto, dove:

$$L_k = \{a_1 \dots a_n \mid a_1 + \dots + a_n \equiv 0 \pmod{k}\}$$

- Sia $D_k = (Q, \Sigma, \delta, q_0, F)$ il DFA definito come:

- $Q = \{q_0, \dots, q_{k-1}\}$
- $F = \{q_0\}$
- $\forall q_i, q_j \in Q$ e $\forall a \in \Sigma$ vale che:

$$\delta(q_i, a) = q_j \iff i + a \equiv j \pmod{k}$$

- Procediamo quindi per induzione su $n \in \mathbb{N}$:

Caso base ($n = 0$):

- Se $w = \varepsilon$, si ha che $\delta^*(q_0, \varepsilon) = q_0$ e che $0 \equiv 0 \pmod{k}$

Ipotesi induttiva:

- Per $n \in \mathbb{N}$, data $w = a_1 \dots a_n \in \Sigma^*$ si ha che:

$$\delta^*(q_0, w) = q_m \iff a_1 + \dots + a_n \equiv m \pmod{k}$$

Passo induttivo:

- Data $w = a_1 \dots a_{n+1} \in \Sigma^*$, siano $q_m, q_h \in Q$ tali che:

$$\delta^*(q_0, a_1 \dots a_n) = q_m \quad \delta(q_m, a_{n+1}) = q_h$$

- Per ipotesi induttiva, si ha che:

$$\delta^*(q_0, a_1 \dots a_n) = q_m \iff a_1 + \dots + a_n \equiv m \pmod{k}$$

inoltre, per come è stata definita δ , abbiamo che:

$$\delta(q_m, a_{n+1}) = q_h \iff m + a_{n+1} \equiv h \pmod{k}$$

- Di conseguenza, concludiamo che:

$$\begin{aligned} \delta^*(q_0, w) = q_h &\iff \delta(\delta^*(q_0, a_1 \dots a_n), a_{n+1}) = q_h \iff \delta(q_m, a_{n+1}) = q_h \\ &\iff m + a_{n+1} \equiv h \pmod{k} \iff a_1 + \dots + a_n + a_{n+1} \equiv h \pmod{k} \end{aligned}$$

- A questo punto, si ha che:

$$w = a_1 \dots a_n \in L(D) \iff \delta^*(q_0, w) \in F \iff \delta^*(q_0, w) = q_0$$

$$\iff a_1 + \dots + a_n \equiv 0 \pmod{k} \iff w \in L$$

implicando quindi che $L_k = L(D_k)$

□

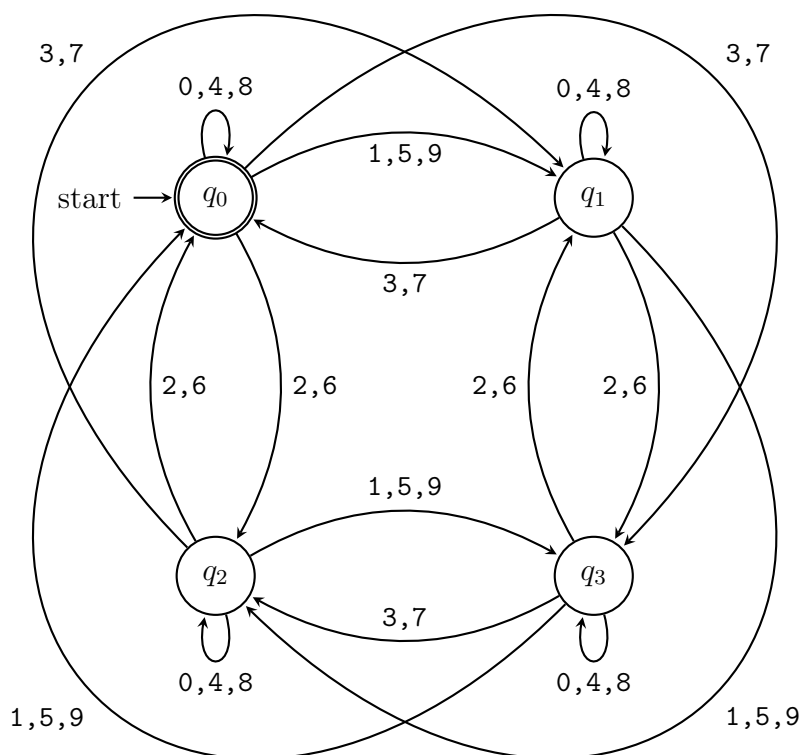


Diagramma del DFA D_4 dettato dalla dimostrazione

Problema 1.11: Linguaggio quozientato

Dati due linguaggi A e B , sia il quoziente su B del linguaggio A definito come:

$$A/B = \{w \in \Sigma^* \mid wx \in A \text{ per qualche } x \in B\}$$

Dimostrare che se $A \in \text{REG}$ allora $A/B \in \text{REG}$

Nota: B può anche non essere regolare

Dimostrazione.

- Dato $A \in \text{REG}$, sia $D = (Q, \Sigma, \delta, q_0, F)$ il DFA tale che $A = L(D)$
- Consideriamo quindi il DFA $D' = \{Q, \Sigma, \delta, q_0, F'\}$, dove:

$$F' = \{q \in Q \mid \exists x \in B, \delta^*(q, x) \in F\}$$

- A questo punto, notiamo che:

$$w \in L(D') \iff \delta^*(q_0, w) \in F' \iff$$

$$\exists x \in B, \delta^*(\delta^*(q_0, w), x) = \delta^*(q_0, wx) \in F \iff w \in A/B$$

concludendo che $A/B = L(D') \in \text{REG}$

□

Problema 1.12

Dato il seguente linguaggio:

$$L = \{0^k w 0^k \mid k \in \mathbb{N}_{>0}, w \in \Sigma^*\}$$

dimostrare che $L \in \text{REG}$

Dimostrazione.

- Consideriamo il linguaggio $L' = \{0w0 \mid w \in \Sigma^*\}$
- Data $x \in L$, si ha che:

$$x \in L \iff \exists k \in \mathbb{N}_{>0}, w \in \Sigma^*, x = 0^k w 0^k$$

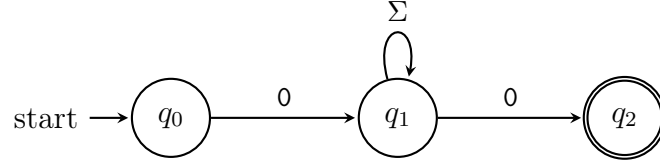
di conseguenza, posto $y = 0^{k-1} w 0^{k-1} \in \Sigma^*$, si ha che $x = 0y0 \in L'$

- Viceversa, dato $x \in L'$, si ha che:

$$x \in L' \iff \exists w \in \Sigma^*, x = 0w0 = 0^1 w 0^1 \implies x \in L$$

- Di conseguenza, concludiamo che $L = L'$

- A questo punto, definiamo il seguente NFA N tale che $L = L' = L(N) \in \text{REG}$:



□

Problema 1.13: Strict-NFA

Uno **Strict-NFA** è una quintupla $S = (Q, \Sigma, \delta, q_0, F)$ la cui funzione di transizione è non deterministica e dove una stringa $w \in \Sigma^*$ viene accettata da S se e solo se ogni ramo di computazione di $S(w)$ termina su uno stato in accettante.

Dimostrare che dato un linguaggio L si ha che:

$$L \in \text{REG} \iff \text{Esiste Strict-NFA che riconosce } L$$

Dimostrazione.

Prima implicazione.

- Dato $L \in \text{REG}$, sia D il DFA tale che $L = L(D)$
- Notiamo che un DFA sia un particolare **Strict-NFA** dotato di un solo ramo di computazione. Di conseguenza, banalmente abbiamo che D stesso sia lo **Strict-NFA** che riconosce L

Seconda implicazione.

- Sia L un linguaggio riconosciuto dallo **Strict-NFA** $S = (Q_S, \Sigma, \delta_S, q_{0_S}, F_S)$
- Consideriamo quindi il DFA $D := (Q_D, \Sigma, \delta_D, q_{0_D}, F_D)$ costruito tramite S stesso:

- $Q_D = \mathcal{P}(Q_S)$
- Dato $R \in Q_D$, definiamo l'estensione di R come:

$$E(R) = \{q \in Q_S \mid \exists p \in R \text{ che raggiunge } q \text{ in } S \text{ tramite solo } \varepsilon\text{-archi}\}$$

- $q_{0_D} = E(\{q_{0_S}\})$
- $F_D = \{R \in Q_D \mid R \subseteq F_S\}$
- Dati $R \in Q_D$ e $a \in \Sigma$, definiamo δ_D come:

$$\delta_D(R, a) = \bigcup_{r \in R} E(\delta_S(r, a))$$

- In particolare, notiamo che tale conversione sia identica alla conversione da NFA a DFA fatta eccezione del modo in cui l'insieme F_D è definito
- Per costruzione stessa di D si ha che:

$$w \in L = L(S) \iff w \in L(D)$$

implicando dunque che $L = L(S) = L(D) \in \text{REG}$.

□

Problema 1.14

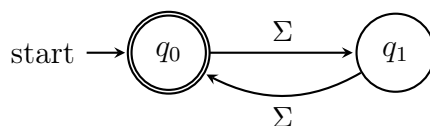
Dato il linguaggio $L = \{w \mid w \text{ termina con } 0 \text{ oppure ha lunghezza pari}\}$, dimostrare che $L \in \text{REG}$

Dimostrazione.

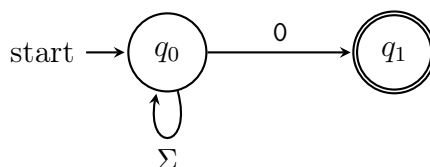
- Prima di tutto, notiamo che il linguaggio L proposto corrisponda all'unione dei seguenti due linguaggi:

$$L = \{w \mid \exists k \in \mathbb{N}, |w| = 2k\} \cup \{w \mid \exists y \in \Sigma^*, w = y0\}$$

- Siano quindi $L_1 = \{w \mid \exists k \in \mathbb{N}, |w| = 2k\}$ e $L_2 = \{w \mid \exists y \in \Sigma^*, w = y0\}$
- Definiamo il DFA D tale che $L_1 = L(D) \in \text{REG}$



- Inoltre, definiamo il NFA N tale che $L_2 = L(N) \in \text{REG}$



- Poiché $L_1, L_2 \in \text{REG}$, per la [Chiusura dell'unione in REG](#) concludiamo che $L \in \text{REG}$

□

- Siano quindi che $w := w_1 \dots w_h$ e $w' := w'_1 \dots w'_h$ tali che $|w| = |w'| = h$ e che $u = ww'$, implicando dunque che $h = \frac{n}{2}$. Per il risultato precedente, ne segue automaticamente che:

$$w_{\frac{1+h}{2}} \neq x_{\frac{1+k}{2}} = a \neq b = x_{\frac{k+1+n}{2}} = w'_{\frac{1+h}{2}} \implies w \neq w' \implies x = ww' \in L$$

- Dunque, abbiamo ottenuto $L = L(G) \in \text{CFL}$
- Il complemento di tale linguaggio risulta essere $\bar{L} = \{ww \mid w \in \{a, b\}^*\}$, il quale abbiamo già dimostrato non essere un linguaggio acontestuale (sezione 2.5). Di conseguenza, concludiamo che $L \in \text{CFL}$, ma $\bar{L} \notin \text{CFL}$

□

2.7 Esercizi svolti

Problema 2.1: Conversione da esp. reg a CFG

Si consideri l'espressione regolare $1\Sigma^*$, dove $\Sigma = \{0, 1\}$. Convertire tale espressione in una grammatica acontestuale e dimostrarne la correttezza

Dimostrazione I.

- Sia $R = 1\Sigma^* = 1(0 \cup 1)^*$
- Sia $G = (V, \Sigma, R, S)$ la CFG definita come:

$$G: \begin{aligned} S &\rightarrow 1A \\ A &\rightarrow 0A \mid 1A \mid \varepsilon \end{aligned}$$

- Dalle regole di G , risulta evidente che se $A \xRightarrow{*} w$ allora $w \in \Sigma^*$
- Procediamo quindi per induzione sulla lunghezza n di $w \in \Sigma^*$:

Caso base ($n = 0$):

- Se $n = 0$, allora $w = \varepsilon$, implicando che $w \in \Sigma^*$ e che $A \Rightarrow w = \varepsilon$

Ipotesi induttiva:

- Per ogni stringa $w \in \Sigma^*$ tale che $|w| = n$, vale che:

$$w \in \Sigma^* \implies A \xRightarrow{*} w$$

Passo induttivo:

- Data una stringa $w = a_1 \dots a_{n+1} \in \Sigma^*$, poiché $|a_2 \dots a_{n+1}| = n$, per ipotesi induttiva si ha che:

$$a_2 \dots a_{n+1} \in \Sigma^* \implies A \xRightarrow{*} a_2 \dots a_{n+1}$$

– A questo punto, notiamo che:

* Se $w_1 = 0$, allora $A \Rightarrow 0A \xRightarrow{*} 0a_2 \dots a_{n+1} = w$

* Se $w_1 = 1$, allora $A \Rightarrow 1A \xRightarrow{*} 1a_2 \dots a_{n+1} = w$

dunque concludiamo che $A \xRightarrow{*} w$

- Di conseguenza, otteniamo che $w \in \Sigma^* \iff A \xRightarrow{*} w$
- Per le regole di G , otteniamo che:

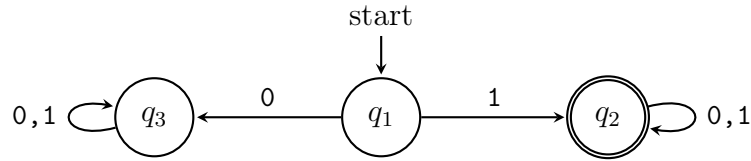
$$w \in L(G) \iff S \xRightarrow{*} w \iff S \Rightarrow 1A \xRightarrow{*} 1y = w \iff w \in L(R)$$

dove $A \xRightarrow{*} y$, implicando che $L(G) = L(R)$

□

Dimostrazione II.

- Sia $R = 1\Sigma^* = 1(0 \cup 1)^*$
- Sia $D = (Q, \Sigma, \delta, q_1, \{q_2\})$ il DFA definito come:



- Notiamo che:

$$w \in L(R) \implies \exists y \in \Sigma^* \mid w = 1y \implies \delta^*(q_0, 1y) = \delta^*(\delta(q_0, 1), y) = \delta^*(q_2, y) = q_2 \implies w \in L(D)$$

e inoltre che:

$$w \notin L(R) \implies \exists y \in \Sigma^* \mid w = 0y \implies \delta^*(q_0, 0y) = \delta^*(\delta(q_0, 0), y) = \delta^*(q_3, y) = q_3 \implies w \notin L(D)$$

- Di conseguenza, si ha che:

$$w \in L(R) \iff w \in L(D)$$

implicando che $L(R) = L(D)$

- A questo punto, tramite la [Conversione da DFA a CFG](#), definiamo la seguente grammatica G tale che $L(G) = L(D)$:

$$G : \begin{array}{l} V_1 \rightarrow 1V_2 \mid 0V_3 \\ V_2 \rightarrow 1V_2 \mid 0V_2 \mid \varepsilon \\ V_3 \rightarrow 1V_3 \mid 0V_3 \end{array}$$

□

Problema 2.2

Mostrare che la seguente grammatica è ambigua:

$$G : \begin{array}{l} S \rightarrow TbT \\ T \rightarrow aTbT \mid bTaT \mid \varepsilon \end{array}$$

Soluzione:

- Ricordiamo che per mostrare che una grammatica sia ambigua dobbiamo dimostrare che esistono due derivazioni sinistre per la stessa parola. Mostriamo quindi di poter derivare in due modi la parola bab .
- Primo modo: dopo aver applicato $S \rightarrow TbT$, applichiamo prima la regola $T \rightarrow bTaT$ sulla T più a sinistra per poi applicare la regola $T \rightarrow \varepsilon$ su tutte le T rimanenti:

$$S \Rightarrow TbT \Rightarrow bTaTbT \Rightarrow baTbT \Rightarrow babT \Rightarrow bab$$

- Secondo modo: dopo aver applicato $S \rightarrow TbT$, applichiamo prima la regola $T \rightarrow \varepsilon$ sulla T più a sinistra per poi applicare la regola $T \rightarrow \varepsilon$ sulla prima T rimanente a sinistra ed infine la regola $T \rightarrow \varepsilon$ su tutte le T rimanenti:

$$S \Rightarrow TbT \Rightarrow bT \Rightarrow baTbT \Rightarrow babT \Rightarrow bab$$

Problema 2.3

Data la seguente grammatica:

$$G : \begin{array}{l} S \rightarrow WbT \\ T \rightarrow aWbT \mid bVaT \mid \varepsilon \\ W \rightarrow aWbW \mid \varepsilon \\ V \rightarrow bVaV \mid \varepsilon \end{array}$$

Mostrare che $aabbbbbaab \in L(G)$. Descrivere un PDA P tale che $L(G) = L(P)$

Soluzione:

- Analizzando la stringa $aabbbbbaab$ e la prima regola $S \rightarrow WbT$, notiamo che una delle b presenti nel centro della stringa debba necessariamente essere dovuta alla b introdotta da tale prima regola.
- A questo punto, notiamo che:

$$W \Rightarrow aWbW \Rightarrow aaWbWbW \xRightarrow{*} aabb$$

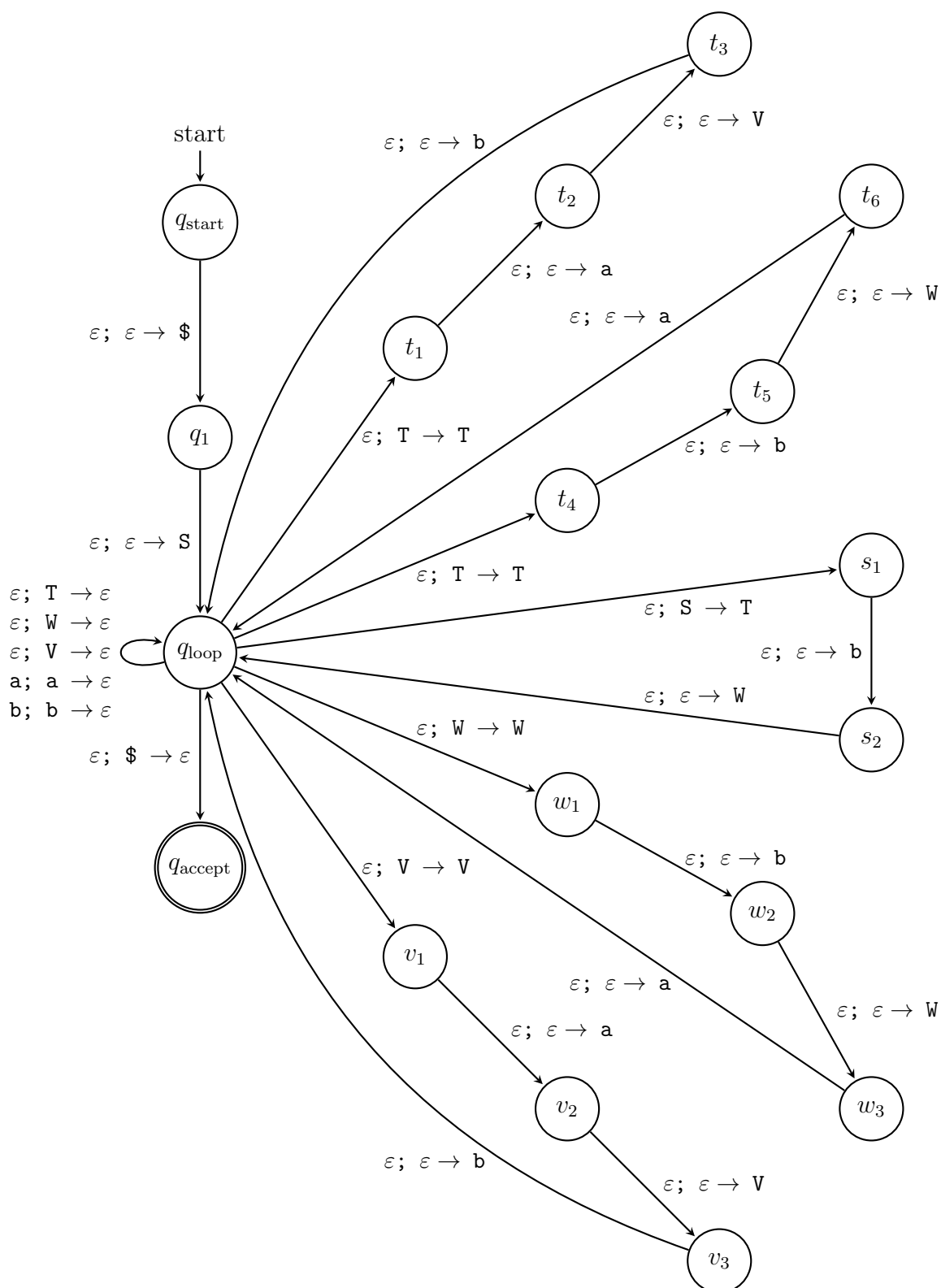
e che:

$$T \Rightarrow bVaT \Rightarrow baT \Rightarrow baaWbT \xRightarrow{*} baab$$

dunque otteniamo che:

$$S \Rightarrow WbT \xRightarrow{*} aabbbT \xRightarrow{*} aabbbbbaab$$

- Tramite la [Conversione da CFG a PDA](#) otteniamo che il PDA P tale che $L(G) = L(P)$ corrisponda a:



Problema 2.4

Classificare ognuno dei seguenti tre linguaggi come (a) regolare, (b) acontestuale ma non regolare, (c) non acontestuale:

1. $L_1 = \{w\#u \mid w, u \in \{0, 1\}^* \text{ e } |w| = |u|\}$
2. $L_2 = \{w \in \{0, 1\}^* \mid |w|_0 \text{ è dispari e } |w|_1 = 1\}$
3. $L_3 = \{w \in \{0, 1, 2\}^* \mid |w|_0 > |w|_1 \text{ e } |w|_0 > |w|_2\}$

Soluzione:

1.
 - Il linguaggio è facilmente riconoscibile da un PDA che memorizza nello stack ogni carattere della prima stringa, per poi rimuovere un carattere dallo stack ogni volta che ne viene letto un simbolo della seconda stringa. Questo conclude che $L_1 \in \text{CFG}$.
 - Utilizziamo il pumping lemma per linguaggi regolari per dimostrare che $L_1 \notin \text{REG}$. Consideriamo la stringa $0^p\#0^p \in L_1$, dove p è la lunghezza del pumping. Siano x, y, z le tre sottostringhe tali che $xyz = 0^p\#0^p$.
 - Per la terza condizione del lemma abbiamo che $|x| \leq p$, dunque $x = 0^k$ per qualche $k \leq p$, mentre $yz = 0^{p-k}\#0^p$. Poiché per la seconda condizione del lemma abbiamo che $|y| > 0$, la sottostringa y deve o contenere solo simboli 0 oppure contenere anche il simbolo $\#$.
 - In entrambi i casi, abbiamo che $xy^0z \notin L_1$, dunque L_1 non può essere regolare. Questo conclude che $L_1 \notin \text{REG}$.
2.
 - Siano $L = \{w \in \{0, 1\}^* \mid w \text{ contiene un numero dispari di } 0\}$ e $L' = \{w \in \{0, 1\}^* \mid w \text{ contiene un singolo } 1\}$.
 - Notiamo che L e L' siano entrambi facilmente riconoscibili da un DFA, dunque $L, L' \in \text{REG}$.
 - Dunque, poiché $L_2 = L \cap L'$ e i linguaggi regolari sono chiusi nell'intersezione, concludiamo che $L_2 \in \text{REG}$.
3.
 - Utilizziamo il pumping lemma per linguaggi acontestuali per dimostrare che $L_3 \notin \text{CFG}$. Consideriamo stringa $0^{p+1}1^p2^p \in L_3$, dove p è il primo numero dispari che sia maggiore della lunghezza del pumping q . Siano u, v, x, y, z le cinque sottostringhe tali che $uvxyz = 0^{p+1}1^p2^p$.
 - Abbiamo tre casi:
 - (a) vxy contiene solo 0. In questo caso, poiché la seconda condizione del lemma impone che $|vy| > 0$, almeno una tra v e y dovrà contenere almeno uno 0, concludendo che il numero di 0 in uv^0xy^0z sia minore di p e dunque che $uv^0xy^0z \notin L_3$
 - (b) vxy non contiene alcun 0. In questo caso, poiché la seconda condizione del lemma impone che $|vy| > 0$, almeno una tra v e y dovrà contenere almeno

un 1 o un 2, concludendo che il numero di 1 o 2 in $uv^i xy^i z$ sia maggiore del numero di 0 (per qualche i sufficientemente grande), dunque $uv^i xy^i z \notin L_3$

- (c) vxy contiene qualche 0 ma non solo. In questo caso, la terza condizione del lemma impone che $|vxy| \leq p$, dunque vxy non può contenere alcun 2. Inoltre, la seconda condizione impone che $|vy| > 0$. Per tanto, se vy contiene più 0 che 1 allora la stringa $uv^0 xy^0 z$ conterrà più 1 che 0, mentre se vy contiene più 1 che 0 allora la stringa $uv^i xy^i z$ per un i sufficientemente grande conterrà più 1 che 0 (ricordiamo che p è dispari per scelta, dunque non esiste in caso in cui abbiano lo stesso numero di 0 e 1). In entrambi i casi, concludiamo che la stringa ottenuta non appartenga a L_3 .

- In tutti e tre i casi, otteniamo che L_3 non possa essere acontestuale. Questo conclude che $L_3 \notin \text{CFG}$.

Problema 2.5

Definire una CFG per il seguente linguaggio:

$$L = \{0^n 1^m 2^p 3^q \mid n, m, p, q \geq 0 \text{ con } n > m \text{ e } p < q\}$$

Soluzione:

- Consideriamo la seguente grammatica G :

$$G : S \rightarrow AB$$

$$A \rightarrow 0A1 \mid 0A \mid 0$$

$$B \rightarrow 2B3 \mid B3 \mid 3$$

- Osserviamo come la regola S divida la generazione della stringa in due parti: la creazione della sottostringa $0^n 1^m$ con $n > m$ e la sottostringa $2^p 3^q$ con $p < q$.
- In particolare, la regola $A \rightarrow 0A1$ garantisce che ogni qual volta venga generato un 1 venga anche generato uno 0 alla sua sinistra. Inoltre, la regola $A \rightarrow 0$ garantisce che la variabile A possa essere eliminata solo aggiungendo uno 0 senza un 1 associato, forzando $n > m$.
- Un'idea speculare viene fornita dalle regole della variabile B . Per tanto, abbiamo che $L(G) = L$.

Problema 2.6

Sia $A = \{a^n b^n \mid n \in \mathbb{N}\}$ e sia $B = \overline{A}$. Definire una grammatica acontestuale per B utilizzando la proprietà di chiusura dei linguaggi regolari rispetto all'unione.

Soluzione:

- Definiamo tre linguaggi L_1, L_2, L_3 disgiunti tra loro:
 - $L_1 = \{a^n b^m \mid n, m \in \mathbb{N}, n < m\}$
 - $L_2 = \{a^n b^m \mid n, m \in \mathbb{N}, n > m\}$
 - $L_3 = L(R)$ dove R è l'espressione regolare $R = (a \cup b)^* ba (a \cup b)^*$
- Affermazione:** $B = L_1 \cup L_2 \cup L_3$

Dimostrazione dell'affermazione

- Risulta evidente che nessuna delle stringhe di L_1, L_2 ed L_3 appartiene ad A , dunque $L_1 \cup L_2 \cup L_3 \subseteq B$
- Consideriamo quindi una stringa $x \in B$. Se $x = a^n b^m$ con $n \neq m$, risulta immediato che $x \in L_1 \cup L_2$.
- Assumiamo quindi che $x \neq a^n b^m$ per ogni $n, m \in \mathbb{N}$. Supponiamo per assurdo che x non contenga la sottostringa ba . Allora, deve necessariamente seguire che $x \in L(a^* b^*)$ poiché non vi possono essere nè alternanze tra le a e le b nè prefissi contenenti delle b , contraddicendo l'assunzione su x . Per tanto, concludiamo necessariamente che $x \in L_3$
- Concludiamo quindi che per ogni $x \in B$ vale che $x \in L_1 \cup L_2 \cup L_3$
- Per i linguaggi L_1 e L_2 definiamo le seguenti grammatiche G_1 :

$$G_1 : S_1 \rightarrow aS_1b \mid S_1b \mid b \quad G_2 : S_2 \rightarrow aS_2b \mid aS_2 \mid a$$

- Per il linguaggio L_3 , invece, definiamo la seguente grammatica G_3 :

$$G_3 : S_3 \rightarrow XbaX \\ X \rightarrow aX \mid bX \mid \varepsilon$$

- Tramite la proprietà di chiusura delle CFG, possiamo definire una grammatica $G = G_1 \cup G_2 \cup G_3$ tale che $B = L(G)$.

$$G : S \rightarrow S_1 \mid S_2 \mid S_3 \\ S_1 \rightarrow aS_1b \mid S_1b \mid b \\ S_2 \rightarrow aS_2b \mid aS_2 \mid a \\ S_3 \rightarrow XbaX \\ X \rightarrow aX \mid bX \mid \varepsilon$$

3.7 Esercizi svolti

Problema 3.1

Descrivere formalmente una TM che riconosca il linguaggio delle stringhe binarie contenenti lo stesso numero di 0 ed 1. Dimostrare la correttezza della soluzione proposta

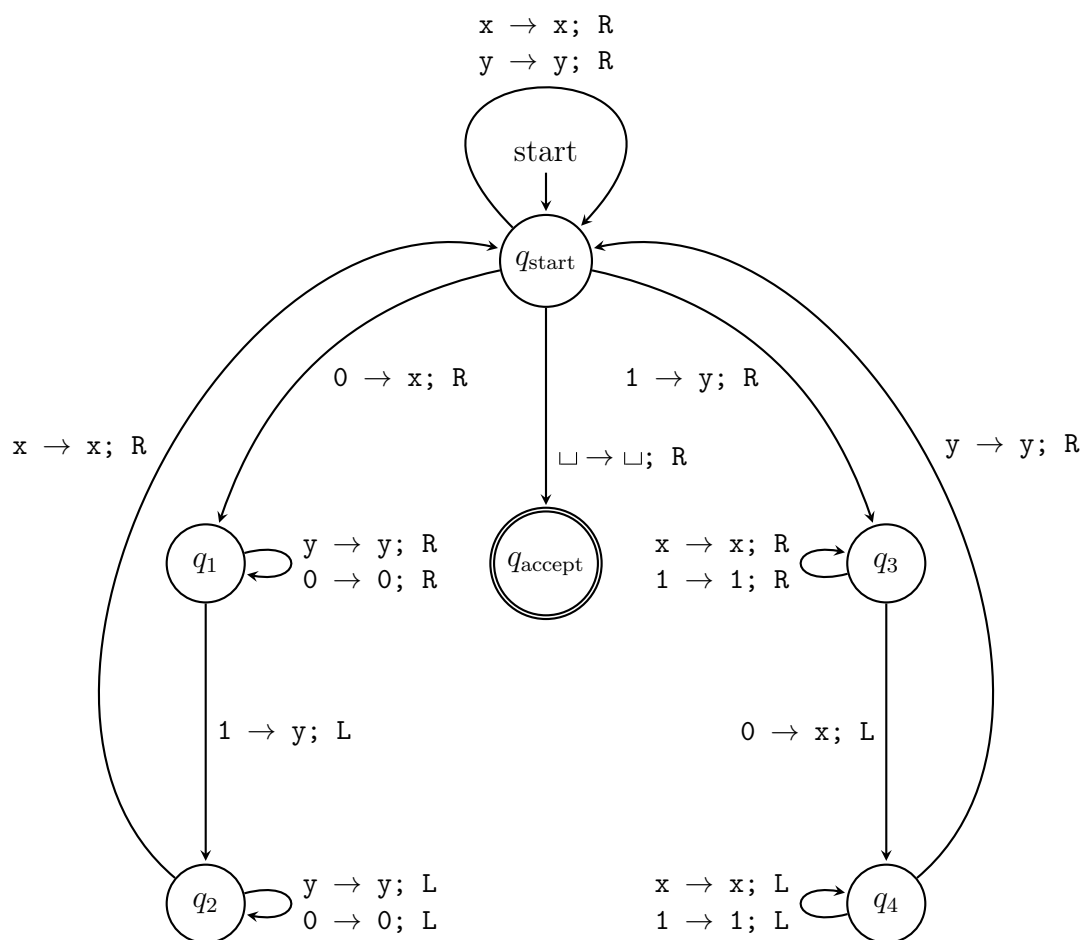
Dimostrazione.

- Sia L il linguaggio richiesto, dove:

$$L = \{w \in \{0, 1\}^* \mid |w|_0 = |w|_1\}$$

- Sia $M = (Q, \Sigma, \Gamma, \delta, q_{\text{start}}, q_{\text{accept}}, q_{\text{reject}})$ la TM definita come:

- $\Sigma = \{0, 1\}$
- $\Gamma = \{0, 1, x, y, \sqcup\}$
- Gli stati di Q e le transizioni di δ sono definiti dal seguente diagramma:



- Notiamo che, se lo stato attuale è q_{start} , si verifica che:
 - Se viene letto uno 0, il cammino $C := q_{\text{start}} \rightarrow q_1 \rightarrow q_2 \rightarrow q_{\text{start}}$ marca tale 0 con una x , per poi scorrere la testina a destra fino al primo 1 presente sul nastro, il quale verrà marcato con una y . Successivamente, la testina scorrerà a sinistra fino alla prima x presente sul nastro, la quale corrisponderà esattamente con lo 0 precedentemente marcato
 - Se viene letto un 1, il cammino $C' := q_{\text{start}} \rightarrow q_3 \rightarrow q_4 \rightarrow q_{\text{start}}$ marca tale 1 con una y , per poi scorrere la testina a destra fino al primo 0 presente sul nastro, il quale verrà marcato con una x . Successivamente, la testina scorrerà a sinistra fino alla prima y presente sul nastro, la quale corrisponderà esattamente con l'1 precedentemente marcato
 - Il cammino $q_{\text{start}} \rightarrow q_{\text{start}}$ assicura che, ad ogni lettura di uno 0, un 1 o un $\sqcup$ non vi siano x o y a destra del simbolo letto
- Supponiamo che $w \in L$. Poiché $|w|_0 = |w|_1$, ad ogni 0 in w corrisponderà un 1 in w . Di conseguenza, la computazione di w su M può percorrere correttamente i cammini C e C' rispettivamente per n ed m volte, tornando sempre allo stato q_{start} . Infine, verrà letto il simbolo $\sqcup$, portando la computazione nello stato q_{accept} , implicando che $w \in L(M)$
- Supponiamo invece che $w \in L(M)$. Essendo una stringa accettata da M , ne segue che la sua computazione percorra correttamente i cammini C e C' per rispettivamente i e j volte. Di conseguenza, poiché ad ogni passaggio su uno dei due cammini vengono marcati uno 0 ed un 1, ne segue che:

$$|w|_0 = i + j = |w|_1 \implies w \in L$$

- Dunque, concludiamo che:

$$w \in L \iff w \in L(M)$$

implicando che $L = L(M)$

□

Problema 3.2: TM a nastro infinito in entrambe le direzioni

Una Bi-TM è una TM in cui il nastro di lavoro è infinito in entrambe le direzioni. Dato un linguaggio $L \subseteq \Sigma^*$ dimostrare che:

$$L \in \text{REC} \iff \exists \text{ Bi-TMM t.c. } L = L(M)$$

Dimostrazione.

Prima implicazione.

- Dato $L \in \text{REC}$, sia M la TM tale che $L = L(M)$
- Poiché una TM è una particolare Bi-TM il cui lato infinito sinistro del nastro non viene mai utilizzato, ne segue automaticamente che essa stessa sia la Bi-TM in grado di riconoscere $L = L(M)$

Seconda implicazione.

- Sia M la Bi-TM tale che $L = L(M)$
- Consideriamo la TM M' definita come:
 $M' = \text{"Data in input la stringa } w\text{:}$
 1. Marca con $\#$ la cella più a sinistra (ossia la prima cella del nastro)
 2. Esegui il programma di M su input w
 3. Ogni volta che l'esecuzione raggiunge la cella contenente $\#$, scorri di una cella a destra tutto il contenuto del nastro e riprendi l'esecuzione"
- Per costruzione stessa di M' , si ha che:

$$w \in L = L(M) \iff w \in L(M')$$

concludendo che $L = L(M) = L(M') \in \text{REC}$

□

Problema 3.3: Insieme di linguaggi distinti

Dato un alfabeto Σ , siano $L_1, \dots, L_k \subseteq \Sigma^*$, dove $k \in \mathbb{N}$, dei linguaggi tali che:

1. $\forall i \neq j \quad L_i \cap L_j = \emptyset$
2. $L_1 \cup \dots \cup L_k = \Sigma^*$
3. $\forall i \in [1, k] \quad L_i \in \text{REC}$, ossia sono Turing-riconoscibili

Dimostrare che $L_1, \dots, L_k \in \text{DEC}$, ossia sono decidibili

Dimostrazione.

- Le prime due proprietà dei linguaggi $L_1, \dots, L_k$ implicano che essi siano una partizione di Σ^*
- Difatti, tramite esse, $\forall i \in [1, k]$ si ha che:

$$w \in \overline{L_i} \iff w \in \Sigma^* - L_i \iff \exists h \neq i \in [1, k], w \in L_h \iff w \in \bigcup_{j \neq i} L_j$$

implicando che:

$$\forall i \in [1, k] \quad \overline{L_i} = \bigcup_{j \neq i} L_j$$

- Poiché $L_1, \dots, L_k \in \text{REC}$, siano $M_1, \dots, M_k$ le TM tali che $\forall i \in [1, k] \quad L_i = L(M_i)$
- Dato $i \in [1, k]$, consideriamo una TM $\overline{M_i}$ definita come:

$\overline{M_i}$ = "Data la stringa w in input:

1. Esegui in parallelo, ossia alternando ad ogni istruzione le loro esecuzioni, i programmi di $M_1, \dots, M_{i-1}, M_{i+1}, \dots, M_k$ con input w
2. Se una delle esecuzioni accetta, $\overline{M_i}$ accetta"

- Per costruzione stessa di $\overline{M_i}$, si ha che:

$$w \in L(\overline{M_i}) \iff \overline{L_i} = \bigcup_{j \neq i} L_j \iff w \in \overline{L_i}$$

implicando che $\overline{L_i} = L(\overline{M_i}) \in \text{REC}$, ossia che L_i sia coTuring-riconoscibile

- Di conseguenza, poiché:

$$L_i \in \text{DEC} \iff L_i \in \text{REC}, L_i \in \text{coREC} \iff L_i \in \text{REC}, \overline{L_i} \in \text{REC}$$

ne segue automaticamente che $L_1, \dots, L_k$ siano decidibili

□

Problema 3.4: Riducibilità al proprio complemento

Dato un linguaggio A tale che $A \leq_m \bar{A}$, dimostrare che:

$$A \in \text{REC} \implies A \in \text{DEC}$$

Dimostrazione.

- Poiché $A \leq_m \bar{A}$, per la **Riducibilità complementare** abbiamo che:

$$A \leq_m \bar{A} \iff \bar{A} \leq_m \bar{\bar{A}} = A$$

- Supponiamo quindi che $A \in \text{REC}$. In tal caso, ne segue che:

$$\bar{A} \leq_m A, A \in \text{REC} \implies \bar{A} \in \text{REC} \implies A \in \text{coREC}$$

- Di conseguenza, poiché $\text{DEC} = \text{REC} \cap \text{coREC}$, concludiamo che:

$$A \in \text{REC} \cap \text{coREC} \iff A \in \text{DEC}$$

□

Problema 3.5: $START-0_{\text{TM}}$ indecidibile

Dato il seguente linguaggio:

$$START-0_{\text{TM}} = \{\langle M \rangle \mid M \text{ TM}, L(M) = \{0y \mid y \in \Sigma^*\}\}$$

dimostrare che è **indecidibile**, ossia che $START-0_{\text{TM}} \notin \text{DEC}$

Dimostrazione.

- Sia $f : \Sigma^* \rightarrow \Sigma^*$ la funzione calcolata dalla seguente TM F definita come:

$F =$ "Data la stringa $\langle M, w \rangle$ in input:

1. Costruisci una TM M' definita come:

$M' =$ "Data la stringa x in input:

- i. Se $x \notin \{0y \mid y \in \Sigma^*\}$, *rifiuta*. Altrimenti, esegui il programma di M su input w
- ii. Se l'esecuzione accetta, allora M' *accetta*, altrimenti *rifiuta*"

2. Restituisci in output la stringa $\langle M' \rangle$ "

- Supponiamo che $w \notin L(M)$. In tal caso, M' rifiuterà qualsiasi stringa, implicando che $L(M') = \emptyset$ e dunque che $\langle M' \rangle \notin START-0_{\text{TM}}$

- Supponiamo ora che $w \in L(M)$. In tal caso, abbiamo che:
 - Se $x \notin \{0w \mid w \in \Sigma^*\}$, allora $x \notin L(M')$
 - Se $x \in \{0w \mid w \in \Sigma^*\}$, allora $x \in L(M')$ poiché $w \in L(M)$

di conseguenza, otteniamo che:

$$x \in L(M') \iff x \in \{0w \mid w \in \Sigma^*\}$$

implicando che $\langle M' \rangle \in START-0_{TM}$

- Di conseguenza, concludiamo che $w \in L(M) \iff L(M') \in START-0_{TM}$
- A questo punto, notiamo che:

$$\langle M, w \rangle \in A_{TM} \iff w \in L(M) \iff f(\langle M, w \rangle) = \langle M' \rangle \in START-0_{TM}$$

implicando quindi che $A_{TM} \leq_m START-0_{TM}$

- Infine, poiché $A_{TM} \notin DEC$, concludiamo che $START-0_{TM} \notin DEC$

□

Problema 3.6: W_{TM} indecidibile

Dato il seguente linguaggio:

$$W_{TM} = \left\{ \langle M, w, a \rangle \mid \begin{array}{l} M = (Q, \Sigma_M, \Gamma_M, \delta, q_s, q_a, q_r) \text{ TM,} \\ w \in \Sigma_M^*, a \in \Gamma_M, \\ M \text{ scrive } a \text{ su input } w \end{array} \right\}$$

dimostrare che è **indecidibile**, ossia che $W_{TM} \notin DEC$

Dimostrazione.

- Sia $f : \Sigma^* \rightarrow \Sigma^*$ la funzione calcolata dalla seguente TM F definita come:

$F =$ "Data la stringa $\langle M, w \rangle$ in input:

1. Costruisci una TM M' definita come:

$M' =$ "Data la stringa x in input:

- i. Esegui il programma di M su input x
- ii. Se l'esecuzione accetta, scrivi a sul nastro, dove a è un simbolo non appartenente all'alfabeto di nastro di M , e *accetta*
- iii. Altrimenti, *rifiuta*

2. Restituisci in output la stringa $\langle M', w, a \rangle$

- Notiamo che:

$$\langle M, w \rangle \in A_{TM} \iff w \in L(M) \iff M'(w) \text{ scrive } a \text{ sul nastro}$$

$$\iff f(\langle M, w \rangle) = \langle M', w, a \rangle \in W_{\text{TM}}$$

implicando quindi che $A_{\text{TM}} \leq_m W_{\text{TM}}$

- Infine, poiché $A_{\text{TM}} \notin \text{DEC}$, concludiamo che $W_{\text{TM}} \notin \text{DEC}$

□

Problema 3.7: $UNION\text{-}ALL_{\text{TM}}$ indecidibile

Dato il seguente linguaggio:

$$UNION\text{-}ALL_{\text{TM}} = \{\langle T, T' \rangle \mid T, T' \text{ TM}, L(T) \cup L(T') = \Sigma^*\}$$

dimostrare che è **indecidibile**, ossia che $UNION\text{-}ALL_{\text{TM}} \notin \text{DEC}$

Dimostrazione.

- Sia $f : \Sigma^* \rightarrow \Sigma^*$ la funzione calcolata dalla seguente TM F definita come:

F = "Data la stringa $\langle M, w \rangle$ in input:

1. Costruisci una TM T definita come:

T = "Data la stringa x in input:

- i. *Rifiuta*"

2. Costruisci una TM T' definita come:

T' = "Data la stringa x in input:

- i. Esegui il programma di M con input w .
- ii. Se l'esecuzione accetta, T' *accetta*, altrimenti *rifiuta*"

3. Restituisci in output la stringa $\langle T, T' \rangle$ "

- Notiamo che:

$$\langle M, w \rangle \in A_{\text{TM}} \iff w \in L(M) \implies L(T) = \emptyset, L(T') = \Sigma^* \implies L(T) \cup L(T') = \Sigma^*$$

$$\iff f(\langle M, w \rangle) = \langle T, T' \rangle \in UNION\text{-}ALL_{\text{TM}}$$

e inoltre che:

$$\langle M, w \rangle \notin A_{\text{TM}} \iff w \notin L(M) \implies L(T) = \emptyset, L(T') = \emptyset \implies L(T) \cup L(T') = \emptyset$$

$$\implies f(\langle M, w \rangle) = \langle T, T' \rangle \notin UNION\text{-}ALL_{\text{TM}}$$

implicando quindi che $A_{\text{TM}} \leq_m UNION\text{-}ALL_{\text{TM}}$

- Infine, poiché $A_{\text{TM}} \notin \text{DEC}$, concludiamo che $UNION\text{-}ALL_{\text{TM}} \notin \text{DEC}$

□

Problema 3.8: ODD_{TM} indecidibile

Dato il seguente linguaggio:

$$ODD_{TM} = \{\langle M \rangle \mid M \text{ TM}, L(M) = \{w \in \Sigma^* \mid |w| \text{ dispari}\}\}$$

dimostrare che è **indecidibile**, ossia che $ODD_{TM} \notin DEC$

Dimostrazione.

- Sia $f : \Sigma^* \rightarrow \Sigma^*$ la funzione calcolata dalla seguente TM F definita come:
 $F =$ "Data la stringa $\langle M, w \rangle$ in input:
 1. Costruisci una TM M' definita come:
 $M' =$ "Data la stringa x in input:
 - i. Se $|x|$ è pari, *rifiuta*. Altrimenti, esegui il programma di M su input w
 - ii. Se l'esecuzione accetta, allora M' *accetta*, altrimenti *rifiuta*"
 2. Restituisci in output la stringa $\langle M' \rangle$ "
- Supponiamo che $w \notin L(M)$. In tal caso, M' rifiuterà qualsiasi stringa, implicando che $L(M') = \emptyset$ e dunque che $\langle M' \rangle \notin ODD_{TM}$
- Supponiamo ora che $w \in L(M)$. In tal caso, abbiamo che:
 - Se $|x|$ è pari, allora $x \notin L(M')$
 - Se $|x|$ è dispari, allora $x \in L(M')$ poiché $M(w)$ accetta
 di conseguenza, otteniamo che:

$$x \in L(M') \iff |x| \text{ dispari}$$

implicando che $\langle M' \rangle \in ODD_{TM}$

- Di conseguenza, concludiamo che $w \in L(M) \iff \langle M' \rangle \in ODD_{TM}$
- A questo punto, notiamo che:

$$\langle M, w \rangle \in A_{TM} \iff w \in L(M) \iff f(\langle M, w \rangle) = \langle M' \rangle \in ODD_{TM}$$

implicando quindi che $A_{TM} \leq_m ODD_{TM}$

- Infine, poiché $A_{TM} \notin DEC$, concludiamo che $ODD_{TM} \notin DEC$

□

Problema 3.9: ODD_{TM} irriconoscibile

Dato il seguente linguaggio:

$$ODD_{TM} = \{\langle M \rangle \mid M \text{ TM}, L(M) = \{w \in \Sigma^* \mid |w| \text{ dispari}\}\}$$

dimostrare che è **irriconoscibile**, ossia che $ODD_{TM} \notin REC$

Dimostrazione.

- Sia $f : \Sigma^* \rightarrow \Sigma^*$ la funzione calcolata dalla seguente TM F definita come:
 $F = \text{"Data la stringa } \langle M, w \rangle \text{ in input:}$
 1. Costruisci una TM M' definita come:
 $M' = \text{"Data la stringa } x \text{ in input:}$
 - i. Se $|x|$ è dispari, *accetta*. Altrimenti, esegui il programma di M su input w
 - ii. Se l'esecuzione accetta, allora M' *accetta*, altrimenti *rifiuta*"
 2. Restituisci in output la stringa $\langle M' \rangle$ "
- Supponiamo che $w \in L(M)$. In tal caso, M' accetterà qualsiasi stringa, implicando che $L(M') = \Sigma^*$ e dunque che $\langle M' \rangle \notin ODD_{TM}$
- Supponiamo ora che $w \notin L(M)$. In tal caso, abbiamo che:
 - Se $|x|$ è dispari, allora $x \in L(M')$
 - Se $|x|$ è pari, allora $x \notin L(M')$ poiché $M(w)$ va in loop o rifiuta
 di conseguenza, otteniamo che:

$$x \in L(M') \iff |x| \text{ dispari}$$

implicando che $\langle M' \rangle \in ODD_{TM}$

- Di conseguenza, concludiamo che $w \in L(M) \iff \langle M' \rangle \notin ODD_{TM}$
- A questo punto, notiamo che:

$$\langle M, w \rangle \in \overline{A_{TM}} \iff \langle M, w \rangle \notin A_{TM} \iff w \notin L(M)$$

$$\iff f(\langle M, w \rangle) = \langle M' \rangle \in ODD_{TM}$$

implicando quindi che $\overline{A_{TM}} \leq_m ODD_{TM}$

- Infine, poiché $\overline{A_{TM}} \notin REC$, concludiamo che $ODD_{TM} \notin REC$

□

Problema 3.10: $DECIDABLE_{TM}$ irriconoscibile

Dato il seguente linguaggio:

$$DECIDABLE_{TM} = \{ \langle M \rangle \mid M \text{ TM}, L(M) \in DEC \}$$

dimostrare che è **irriconoscibile**, ossia che $DECIDABLE_{TM} \notin REC$

Dimostrazione.

- Sia $f : \Sigma^* \rightarrow \Sigma^*$ la funzione calcolata dalla seguente TM F definita come:
 $F =$ "Data la stringa $\langle M, w \rangle$ in input:
 1. Costruisci una TM M' definita come:

$M' =$ "Data la stringa x in input:

 - i. Esegui il programma di M su input w .
 - ii. Se l'esecuzione rifiuta, M' *rifiuta*
 - iii. Altrimenti, interpreta $x = \langle M'', y \rangle$ ed esegui M'' su input y .
 - iv. Se l'esecuzione accetta, M' *accetta*. Altrimenti, M' *rifiuta*.
 2. Restituisci in output la stringa $\langle M' \rangle$ "
- Supponiamo che $\langle M, w \rangle \in A_{TM}$. In tal caso, la macchina M' salterà le prime due istruzioni, comportandosi come un normalissimo riconoscitore di A_{TM} , linguaggio che sappiamo essere indecidibile.

$$\langle M, w \rangle \in A_{TM} \implies L(M) = A_{TM} \implies f(\langle M, w \rangle) = \langle M' \rangle \notin DECIDABLE_{TM}$$

- Viceversa, se $\langle M, w \rangle \notin A_{TM}$ allora l'esecuzione $M(w)$ all'interno di M' andrà o in loop o rifiuterà. In entrambi i casi, M' non accetterà alcuna stringa, dunque $L(M) = \emptyset$, linguaggio decidibile da una macchina che rifiuta sempre.

$$\langle M, w \rangle \notin A_{TM} \implies L(M) = \emptyset \implies f(\langle M, w \rangle) = \langle M' \rangle \in DECIDABLE_{TM}$$

- Di conseguenza, abbiamo che:

$$\langle M, w \rangle \in \overline{A_{TM}} \iff \langle M, w \rangle \in A_{TM} \iff f(\langle M, w \rangle) = \langle M' \rangle \in DECIDABLE_{TM}$$

implicando quindi che $\overline{A_{TM}} \leq_m DECIDABLE_{TM}$. Poiché $\overline{A_{TM}} \notin REC$, concludiamo che $DECIDABLE_{TM} \notin REC$

□

4.13 Esercizi svolti

Problema 4.1: Riducibilità quasi-polinomiale

Sia QP la classe dei linguaggi decidibili in tempo quasi-polinomiale, definita come:

$$\text{QP} = \bigcup_{k=1}^{\infty} \text{DTIME}(n^{\log^k n})$$

Dati due linguaggi A e B tali che $A \leq_m^P B$, stabilire se la seguente affermazione sia vera o falsa:

$$B \in \text{QP} \implies A \in \text{QP}$$

Dimostrazione.

- Sia $f : \Sigma^* \rightarrow \Sigma^*$ la funzione tramite cui si ha che $A \leq_m^P B$, implicando che $\exists k \in \mathbb{N}$ tale che f sia computabile in $O(n^k)$
- Dato $B \in \text{QP}$, sia D_B il decisore che decide B in tempo $O(n^{\log^j n})$, dove $j \in \mathbb{N}$
- Notiamo quindi che:

$$\lim_{n \rightarrow +\infty} \frac{n^k}{n^{\log^j n}} = \lim_{n \rightarrow +\infty} n^{k - \log^j n} = 0$$

poiché $k - \log^j n$ tende a $-\infty$ per $n \rightarrow +\infty$, implicando che $n^k = o(n^{\log^j n})$ e dunque che $n^k = O(n^{\log^j n})$

- Sia quindi D_A la TM definita come:

$D_A =$ "Data la stringa w in input:

1. Calcola $f(w)$
2. Esegui il programma di D_B su input $f(w)$.
3. Se l'esecuzione accetta, *accetta*. Altrimenti, *rifiuta*"

- Risulta evidente che:

$$w \in L(D_A) \iff f(w) \in L(D_B) = B$$

implicando che $A = L(D_A) \in \text{DEC}$.

- Inoltre, poiché ogni istruzione di D_A è eseguibile in tempo $O(n^{\log^j n})$, concludiamo che $A = L(D_A) \in \text{DTIME}(n^{\log^j n}) \subseteq \text{QP}$
- Dunque, stabiliamo che l'affermazione sia vera

□

Problema 4.2: $P = NP = NP\text{-Complete}$

Escludendo i linguaggi $\emptyset$ e Σ^* da qualsiasi classe, dimostrare che se $P = NP$ allora $NP = NP\text{-Complete}$

Nota: l'esclusione di $\emptyset$ e Σ^* è necessaria in quanto per definizione stessa di riduzione tramite mappatura sia impossibile che un linguaggio sia riducibile a $\emptyset$ o Σ^* o che quest'ultimi siano riducibili ad un altro linguaggio

Dimostrazione.

- Osserviamo che $NP\text{-Complete} \subseteq NP\text{-Complete} = NP \cap NP\text{-Hard}$, dunque si ha che $NP\text{-Complete} \subseteq NP$
- Consideriamo quindi $B \in NP = P$ tale che $B \neq \emptyset, \Sigma^*$. Poiché $B \neq \emptyset, \Sigma^*$, esistono necessariamente due stringhe $x, y \in \Sigma^*$ tali che $x \in B$ e $y \notin B$ (se non assumessimo ciò, una delle due stringhe non esisterebbe).
- Dato un qualsiasi linguaggio $A \in NP = P$, sia D_A il decisore in tempo polinomiale tale che $A = L(D_A)$
- Sia quindi $f : \Sigma^* \rightarrow \Sigma^*$ la funzione calcolata dalla seguente TM F :

$F =$ "Data la stringa w in input:

1. Esegui il programma di D_A su input w
2. Se l'esecuzione accetta, restituisci in output la stringa x
3. Altrimenti, restituisci in output la stringa y "

- In particolare, notiamo che poiché D_A sia un decisore, la TM F termini sempre l'esecuzione. Di conseguenza, non nascono problemi riguardo la calcolabilità di f
- Notiamo quindi che:

$$w \in A = L(D_A) \implies f(w) = x \in B$$

e inoltre che:

$$w \notin A = L(D_A) \implies f(w) = y \notin B$$

- Poiché l'esecuzione di D_A viene svolta in tempo polinomiale, ne segue che f sia calcolabile in tempo polinomiale, implicando che

$$\forall A \in NP = P \quad A \leq_m^P B$$

concludendo che $B \in NP\text{-Complete}$

□

Problema 4.3: Padding argument

Dimostrare che se $P = NP$ allora $EXP = NEXP$

Dimostrazione.

- Sappiamo già che $EXP \subseteq NEXP$, quindi è sufficiente dimostrare che in tal caso si ha che $NEXP \subseteq EXP$. Dato $L \in NEXP$, sappiamo che esiste una NTM N che decide L in tempo esponenziale.
- Consideriamo quindi il seguente linguaggio:

$$L' = \{x1^{2^{|x|^c}} \mid x \in L\}$$

dove c è una costante arbitraria.

- Sia N' la NTM definita come:

$N' =$ "Data la stringa y in input:

1. Genera una stringa casuale x .
2. Verifica che $y = x1^{2^{|x|^c}}$. Se falso, *rifiuta*.
3. Simula N su input x . Se la simulazione accetta, *accetta*. Altrimenti, *rifiuta*."

Dove per definizione stessa di N' risulta evidente che $L(N') = L'$.

- A questo punto, è opportuno notare che la dimensione dell'input di N' sia di per se esponenziale. In particolare, per ogni stringa $x \in L$ tale che $|x| = n$, l'input di N' avrà lunghezza $m = n + 2^{n^c}$. Per tanto, poiché la simulazione di N su x richiede tempo esponenziale 2^{n^d} , dove d è un'altra costante, tale simulazione richiede $O(m^k)$ passi, dove $k = \max(c, d)$, dunque è polinomiale rispetto alla dimensione dell'input. Ciò conclude che N' sia un decisore non-deterministico per L' .
- Poiché in ipotesi si ha che $P = NP$, deve esistere un decisore di tempo polinomiale M' tale che $L(M') = L'$. Definiamo quindi la seguente TM:

$M =$ "Data la stringa x in input:

1. Simula M' su input $x1^{2^{|x|^c}}$. Se la simulazione accetta, *accetta*. Altrimenti, *rifiuta*.

Dove per definizione stessa di N' risulta evidente che $L(M) = L$. Inoltre, dato $|x| = n$, la simulazione di M' richiede tempo esponenziale rispetto alla dimensione dell'input, dunque concludiamo che $L \in EXP$.

□

Problema 4.4: Chiusure di NL

Dimostrare che NL è chiusa rispetto alle operazioni di unione, intersezione e star di Kleene

Dimostrazione.

- Dati due linguaggi $A, B \in \text{NL}$, siano V_A e V_B i rispettivi verificatori in spazio $O(\log n)$ per A e B
- Sia quindi V_U la TM definita come:

$V_U =$ "Data la stringa $\langle w, c \rangle$ in input:

1. Esegui il programma di V_A su input $\langle w, c \rangle$. Se l'esecuzione accetta, V_U accetta
2. Esegui il programma di V_B su input $\langle w, c \rangle$. Se l'esecuzione accetta, V_U accetta
3. Altrimenti, *rifiuta*"

- Per costruzione di V_U , risulta evidente che:

$$w \in A \cup B \iff w \in A \vee w \in B \iff$$

$$\exists c \in \Sigma^*, \langle w, c \rangle \in L(V_A) \vee \langle w, c \rangle \in L(V_B) \iff \langle w, c \rangle \in L(V_U)$$

implicando che V_U sia un verificatore per $A \cup B$. Inoltre, poiché sia l'esecuzione di V_A che l'esecuzione di V_B richiedono spazio $O(\log n)$, concludiamo che $A \cup B \in \text{NL}$

- Analogamente, sia V_I la TM definita come:

$V_I =$ "Data la stringa $\langle w, c \rangle$ in input:

1. Esegui il programma di V_A su input $\langle w, c \rangle$. Se l'esecuzione rifiuta, V_I rifiuta
2. Esegui il programma di V_B su input $\langle w, c \rangle$. Se l'esecuzione rifiuta, V_I rifiuta
3. Altrimenti, *accetta*"

- Per costruzione di V_I , risulta evidente che:

$$w \in A \cap B \iff w \in A \wedge w \in B \iff$$

$$\exists c \in \Sigma^*, \langle w, c \rangle \in L(V_A) \wedge \langle w, c \rangle \in L(V_B) \iff \langle w, c \rangle \in L(V_I)$$

implicando che V_I sia un verificatore per $A \cap B$. Inoltre, poiché sia l'esecuzione di V_A che l'esecuzione di V_B richiedono spazio $O(\log n)$, concludiamo che $A \cap B \in \text{NL}$

- Sia invece N la NTM che decide A in tempo $O(\log n)$
- Sia quindi S la TM definita come:

$S =$ "Data la stringa $\langle w, c \rangle$ in input:

1. Calcola $|w| = n$
2. Scegli non deterministicamente un valore $k \in [1, n]$

3. Scegli non deterministicamente una partizione di w , ossia delle stringhe $w_1, \dots, w_k$ tali che $w = w_1 \dots w_k$
4. Ripeti le seguenti istruzioni per $i = 1, \dots, k$:
 5. Esegui N su input w_i . Se N rifiuta allora S rifiuta
 6. Altrimenti, pulisci lo spazio utilizzato ed esegui la prossima iterazione sullo stesso spazio
7. Accetta"

- Per costruzione stessa di S , si ha che:

$$w \in L(S) \iff \text{Esiste un ramo che accetta } w \iff$$

$$\text{Esiste una partizione } w_1, \dots, w_k \text{ tale che } \forall i \in [1, k] \ w_i \in L(N) = A$$

$$\iff w = w_1 \dots w_k \in A^*$$

implicando che $L(S) = A^*$. Inoltre, poiché l'esecuzione di N richiede spazio $O(\log n)$ ed S utilizza solo dei contatori, lo spazio richiesto da S risulta essere $O(\log n)$, concludendo $A^* = L(S) \in \text{NL}$

□

Problema 4.5

Dimostrare che per ogni $k \in \mathbb{N}$ si abbia che $\text{NTIME}(n^k) \subsetneq \text{PSPACE}$

Stabilire se tale risultato sia sufficiente a dire che $\text{NP} \subsetneq \text{PSPACE}$

Dimostrazione.

- Tramite il teorema del [Tempo limitante lo spazio](#) e il [Teorema di Savitch](#), si ha che:

$$\forall k \in \mathbb{N} \quad \text{NTIME}(n^k) \subseteq \text{NSPACE}(n^k) \subseteq \text{DSPACE}(n^{2k})$$

- Inoltre, poiché $n^{2k} = o(n^{2k+1})$, per la [Distinzione spaziale tra le classi](#), si ha che:

$$\forall k \in \mathbb{N} \quad \text{NTIME}(n^k) \subseteq \text{DSPACE}(n^{2k}) \subsetneq \text{DSPACE}(n^{2k+1}) \subseteq \text{PSPACE}$$

- Controintuitivamente, il risultato non è sufficiente a stabilire che $\text{NP} \subsetneq \text{PSPACE}$. Difatti, per stabilire ciò è necessario dimostrare che:

$$\forall k, j \in \mathbb{N} \quad \text{NTIME}(n^k) \subsetneq \text{DSPACE}(n^j)$$

□

Problema 4.6: Problema della soddisfacibilità delle 2CNF

Dato il seguente linguaggio:

$$2SAT = \{\langle \phi \rangle \mid \phi \text{ è una 2CNF soddisfacibile}\}$$

dimostrare che $2SAT \in P$

Dimostrazione.

- Prima di procedere con la dimostrazione, forniamo l'intuizione alla base della dimostrazione stessa:

– Dati due letterali α e β si ha che:

$$(\alpha \vee \beta) \equiv (\bar{\alpha} \implies \beta) \equiv (\bar{\beta} \implies \alpha)$$

- A questo punto, sia P il decisore polinomiale tale che $L(P) = PATH \in P$
- Sia inoltre M la TM definita come:

$M =$ "Data la stringa $\langle \phi \rangle$ in input:

1. Costruisci un grafo $G = (V, E)$ definito come:
 - Per ogni variabile x in ϕ , crea i nodi x e $\bar{x}$
 - Per ogni clausola $(\alpha \vee \beta)$ in ϕ , dove α e β sono dei letterali (ossia una variabile o una sua negazione), crea gli archi $(\bar{\alpha}, \beta)$ e $(\bar{\beta}, \alpha)$
2. Ripeti le seguenti istruzioni per ogni variabile x in ϕ :
 3. Esegui P su input $\langle G, x, \bar{x} \rangle$
 4. Esegui P su input $\langle G, \bar{x}, x \rangle$
 5. Se entrambe le esecuzioni accettano, *rifiuta*
 6. *Accetta*"

- Consideriamo quindi una 2CNF ϕ e il grafo G ad essa associato.
- Notiamo che dati due letterali α e β , si abbia che:

$$\text{Esiste cammino } \alpha \rightarrow \beta \text{ in } G \iff \exists (\alpha, \gamma_1), (\gamma_1, \gamma_2), \dots, (\gamma_k, \beta) \in E$$

$$\iff (\bar{\alpha} \vee \gamma_1) \wedge (\bar{\gamma_1} \vee \gamma_2) \wedge \dots \wedge (\bar{\gamma_k} \vee \beta) \text{ sottoformula di } \phi$$

$$\iff (\alpha \implies \gamma_1) \wedge (\gamma_1 \implies \gamma_2) \wedge \dots \wedge (\gamma_k \implies \beta) \text{ sottoformula di } \phi$$

$$\iff \text{Per } \phi \text{ vale l'implicazione } (\alpha \implies \beta)$$

- Supponiamo quindi per assurdo che ϕ sia soddisfacibile e che esista una variabile x_i in ϕ per cui esistano i cammini $x \rightarrow \bar{x}$ e $\bar{x} \rightarrow x$ in G

- Sia quindi $\phi(x_1, \dots, x_i, \dots, x_m)$ un assegnamento che soddisfa ϕ :
 - Se $x_i = \text{True}$, tramite il cammino $x_i \rightarrow \bar{x}_i$ ne segue che affinché ϕ rimanga soddisfacibile sia necessario che l'implicazione $x_i \implies \bar{x}_i$ sia vera, il che è possibile solo se $\bar{x}_i = \text{True}$, implicando che $x_i = \bar{x}_i$, portando dunque ad un assurdo
 - Se $\bar{x}_i = \text{True}$, tramite il cammino $\bar{x}_i \rightarrow x_i$ ne segue che affinché ϕ rimanga soddisfacibile sia necessario che l'implicazione $\bar{x}_i \implies x_i$ sia vera, cosa possibile solo se $x_i = \text{True}$, implicando che $x_i = \bar{x}_i$, portando dunque ad un assurdo

- Di conseguenza, ne segue necessariamente che:

$$\phi \text{ soddisfacibile} \implies \nexists \text{ variabile } x \text{ in } \phi \text{ per cui } x \rightarrow \bar{x} \text{ e } \bar{x} \rightarrow x$$

- Viceversa, supponiamo che tale variabile x_i non esista. Di conseguenza, per ogni coppia di letterali α e β ne segue che se $\alpha \rightarrow \beta$ in G , l'implicazione $(\alpha \implies \beta)$ valida in ϕ è sempre soddisfacibile a vuoto.
- Di conseguenza, esisterà sempre un assegnamento $\phi(x_1, \dots, x_m)$ in grado di soddisfare ϕ , implicando che:

$$\nexists \text{ variabile } x \text{ in } \phi \text{ per cui } x \rightarrow \bar{x} \text{ e } \bar{x} \rightarrow x \implies \phi \text{ soddisfacibile}$$

- A questo punto, per costruzione di M , ne segue che:

$$\langle \phi \rangle \in 2SAT \iff \nexists \text{ variabile } x \text{ in } \phi \text{ per cui } x \rightarrow \bar{x} \text{ e } \bar{x} \rightarrow x \iff \langle \phi \rangle \in L(M)$$

implicando che $2SAT = L(M)$

- Inoltre, poiché $L(P) = PATH \in \mathbf{P}$ e poiché il grafo è costruibile in tempo polinomiale, concludiamo che M sia un decisore polinomiale e dunque che $2SAT = L(M) \in \mathbf{P}$

□

Problema 4.7: Strongly Non deterministic TM

Una Strongly Non deterministic TM (SNTM) è una TM S dotata di tre possibili stati finali: *accept*, *reject* e *not sure*.

In particolare, una SNTM S decide il linguaggio $L(S)$ nel seguente modo:

- Se $w \in L(S)$ allora tutti i rami dell'albero di computazione di S su w terminano con *accept* o *not sure*, dove almeno uno di tali rami termina su *accept*
- Se $w \notin L(S)$ allora tutti i rami dell'albero di computazione di S su w terminano con *reject* o *not sure*, dove almeno uno di tali rami termina su *reject*

Dimostrare che dato un linguaggio L si ha che:

$$L \in \text{NP} \cap \text{coNP} \iff \text{Esiste SNTM che decide } L \text{ in tempo polinomiale}$$

Dimostrazione.

Prima implicazione.

- Dato $L \in \text{NP} \cap \text{coNP}$, si ha che $L, \bar{L} \in \text{NP}$
- Siano quindi $N, \bar{N}$ le due NTM che decidono rispettivamente L e $\bar{L}$ in tempo polinomiale
- Sia inoltre S la SNTM definita come:

$S =$ "Data la stringa w in input:

1. Esegui non deterministicamente un ramo di N su input w
 2. Esegui non deterministicamente un ramo di $\bar{N}$ su input w
 3. Se l'esecuzione del ramo di N accetta e l'esecuzione del ramo di $\bar{N}$ rifiuta, allora S termina su *accept*
 4. Se l'esecuzione del ramo di N rifiuta e l'esecuzione del ramo di $\bar{N}$ accetta, allora S termina su *reject*
 5. Altrimenti, S termina su *not sure*
- In particolare, notiamo che per ogni ramo di esecuzione di S si abbia che:
 - Se il ramo di N accetta e il ramo di $\bar{N}$ rifiuta, allora $w \in L(N) = L$ e S termina su *accept*
 - Se il ramo di N rifiuta e il ramo di $\bar{N}$ accetta, allora $w \in \bar{L} \iff w \notin L$ e S termina su *reject*
 - Se il ramo di N rifiuta e il ramo di $\bar{N}$ rifiuta, non possiamo dire nulla sull'appartenenza di w ad uno dei due linguaggi e S termina su *not sure*
 - È impossibile che sia il ramo di N che il ramo di $\bar{N}$ accettino, poiché ciò implicherebbe che $w \in L \cap \bar{L} = \emptyset$

- Supponiamo quindi che $w \in L(S)$. Per definizione, ne segue che esista un ramo di S che accetta e nessun ramo di S che rifiuta, implicando che esista un ramo di N che accetta w e dunque che $w \in L$
- Viceversa, se $w \in L(S)$ allora per definizione, ne segue che esista un ramo di S che rifiuta e nessun ramo di S che accetta, implicando che esista un ramo di $\overline{N}$ che accetta w e dunque che $w \notin L$
- Di conseguenza, concludiamo che:

$$w \in L(S) \iff w \in L$$

implicando che $L(S) = L$

- Inoltre, poiché N e $\overline{N}$ sono decisi in tempo polinomiale, ogni ramo di S avrà tempo di esecuzione polinomiale, concludendo che anche S decida L in tempo polinomiale

Seconda implicazione.

- Supponiamo che esista una SNTM S che decide un linguaggio L in tempo polinomiale
- Sia N la NTM definita come:

$N =$ "Data la stringa w in input:

1. Esegui non deterministicamente un ramo di S su input w
2. Se l'esecuzione del ramo di S accetta, allora N accetta. Altrimenti, N rifiuta"

- Per costruzione stessa di N , ne segue che:

$$w \in L(S) \iff \text{Esiste un ramo di } S \text{ che accetta e nessun ramo che rifiuta}$$

$$\implies \text{Esiste almeno un ramo di } N \text{ che accetta } w \iff w \in L(N)$$

e inoltre che:

$$w \notin L(S) \iff \text{Esiste un ramo di } S \text{ che rifiuta e nessun ramo che accetta}$$

$$\implies \text{Non esiste un ramo di } N \text{ che accetta } w \iff w \notin L(N)$$

dunque concludiamo che $L = L(S) = L(N)$. Inoltre, poiché l'esecuzione di S è polinomiale, ne segue che anche N decida L in tempo polinomiale, implicando che $L \in \text{NP}$

- Analogamente, sa $\overline{N}$ la NTM definita come:

$\overline{N} =$ "Data la stringa w in input:

1. Esegui non deterministicamente un ramo di S su input w
2. Se l'esecuzione del ramo di S rifiuta, allora N accetta. Altrimenti, N rifiuta"

- Per costruzione stessa di N , ne segue che:

$w \in L(S) \iff$ Esiste un ramo di S che accetta e nessun ramo che rifiuta

$\implies$ Non esiste un ramo di N che accetta $w \iff w \notin L(N)$

e inoltre che:

$w \notin L(S) \iff$ Esiste un ramo di S che rifiuta e nessun ramo che accetta

$\implies$ Esiste almeno un ramo di N che accetta $w \iff w \in L(N)$

dunque concludiamo che $\overline{L} = \overline{L(S)} = L(N)$. Inoltre, poiché l'esecuzione di S è polinomiale, ne segue che anche N decida $\overline{L}$ in tempo polinomiale, implicando che $\overline{L} \in \text{NP}$

- Di conseguenza, poiché $L, \overline{L} \in \text{NP}$, concludiamo che $L \in \text{NP} \cap \text{coNP}$

□

Problema 4.8

Dato il seguente linguaggio:

$$\text{min-}k\text{-PATH} = \left\{ \langle G, s, t, k \rangle \mid \begin{array}{l} G = (V_G, E_G) \text{ grafo non diretto con} \\ \text{cammino } s \rightarrow t \text{ con minimo } k \text{ archi} \end{array} \right\}$$

dimostrare che $\text{min-}k\text{-PATH} \in \text{NP}$

Dimostrazione.

- Sia V la TM definita come:

$V =$ "Data in input la stringa $\langle \langle G, s, t, k \rangle, P \rangle$:

1. Interpreta $P = v_1, \dots, v_h$, dove $v_1, \dots, v_h \in V_G$. Se falso, *rifiuta*.
2. Verifica che $v_1 = s$ e che $v_h = t$. Se falso, *rifiuta*.
3. Verifica che $h \geq k$. Se falso, *rifiuta*.
4. Verifica che $v_i \neq v_j$ per ogni $i \neq j$. Se falso, *rifiuta*.
5. Verifica che per ogni $i = 1, \dots, h-1$ si abbia che $(v_i, v_{i+1}) \in E_G$. Se falso, *rifiuta*.
6. *Accetta*.

- Chiaramente, dalla definizione stessa di V risulta evidente che esso sia un verificatore per $\text{min-}k\text{-PATH}$. Inoltre, ogni operazione svolta da V è eseguibile in tempo polinomiale, dunque concludiamo $L \in \text{NP}$.

□

Problema 4.9: $3COL \leq_m^P 4COL$

Dato un grafo G , diciamo che G è k -colorabile se dati k colori è possibile marcare ogni nodo di G con uno dei k colori in modo che non esistano due nodi adiacenti con lo stesso colore assegnato.

Dati i seguenti due linguaggi:

$$3COL = \{\langle G \rangle \mid G \text{ grafo 3-colorabile}\}$$

$$4COL = \{\langle G \rangle \mid G \text{ grafo 4-colorabile}\}$$

dimostrare che $3COL \leq_m^P 4COL$

Dimostrazione.

- Sia $f : \Sigma^* \rightarrow \Sigma^*$ la funzione calcolata dalla seguente TM F definita come:
 $F = \text{"Data la stringa } \langle G \rangle \text{ in input, dove } G = (V, E):$
 1. Aggiungi un nodo v a G
 2. Per ogni nodo u in G diverso da v , aggiungi un arco (u, v)
 3. Restituisci in output la stringa $\langle G' \rangle$, dove G' è il grafo modificato"
- Supponiamo quindi che $\langle G \rangle \in 3COL$, implicando che $\forall v_1, v_2, v_3 \in V$ si abbia che (v_1, v_2, v_3) è 3-colorabile.
- Poiché il nodo v aggiunto è adiacente ad ogni altro nodo in V , assegnando ad esso un quarto colore risulta evidente che (v_1, v_2, v_3, v) è 4-colorabile, implicando che $\langle G' \rangle \in 4COL$
- Supponiamo invece che $\langle G \rangle \notin 3COL$, implicando che $\exists v_1, v_2, v_3 \in V$ per cui (v_1, v_2, v_3) non è 3-colorabile
- Consideriamo quindi la quadrupla (v_1, v_2, v_3, v) . Assegnando un quarto colore a v , la quadrupla non sarà 4-colorabile poiché (v_1, v_2, v_3) non è 3-colorabile, implicando che (v_1, v_2, v_3, v) non è 4-colorabile, dunque che $\langle G' \rangle \notin 4COL$
- Dunque, ne segue che $\langle G \rangle \in 3COL \iff f(\langle G \rangle) = \langle G' \rangle \in 4COL$. Inoltre, poiché l'aggiunta del nodo v e degli m archi, dove $m = |V|$, richiede tempo polinomiale, concludiamo che $3COL \leq_m^P 4COL$

□

Problema 4.10

Dato un grafo G , diciamo che G è k -colorabile se dati k colori è possibile marcare ogni nodo di G con uno dei k colori in modo che non esistano due nodi adiacenti con lo stesso colore assegnato.

Dato il seguente linguaggio:

$$4COL = \{\langle G \rangle \mid G \text{ grafo 4-colorabile}\}$$

dimostrare che $4COL \leq_m^P SAT$

Dimostrazione.

- Assumiamo che i quattro colori utilizzati siano rosso, verde, blu e giallo. Per ogni nodo $v \in V(G)$, definiamo le variabili v_r, v_g, v_b, v_y , dove ad esempio $v_r = \text{True}$ se e solo se v viene colorato di rosso.
- Per ogni nodo $v \in V(G)$, definiamo la seguente formula:

$$\phi_v = (v_r \vee v_g \vee v_b \vee v_y)$$

Intuitivamente, tale formula esprime che il nodo v debba essere di almeno uno dei 4 colori disponibili. Per ogni arco $(v, u) \in E(G)$, invece, definiamo la seguente formula:

$$\phi_{(v,u)} = (\overline{v_r} \vee \overline{u_r}) \wedge (\overline{v_g} \vee \overline{u_g}) \wedge (\overline{v_b} \vee \overline{u_b}) \wedge (\overline{v_y} \vee \overline{u_y})$$

Tale formula esprime che i nodi v, u appartenenti all'arco (v, u) devono avere colore diverso. Infine, definiamo quindi la seguente formula:

$$\phi_G = \left(\bigwedge_{v \in V(G)} \phi_v \right) \wedge \left(\bigwedge_{(v,u) \in E(G)} \phi_{(v,u)} \right)$$

Per costruzione stessa, risulta evidente che G sia 4-colorabile se e solo se ϕ_G è soddisfacibile.

- Sia quindi $f : \Sigma^* \rightarrow \Sigma^*$ la funzione calcolata dalla seguente TM F :

$F = \text{"Data la stringa } \langle G \rangle \text{ in input:}$

1. Costruisci $\langle \phi \rangle_G$ e restituiscila"

Chiaramente, abbiamo che $\langle G \rangle \in 4COL \iff f(\langle G \rangle) = \langle \phi_G \rangle \in SAT$.

- Siano quindi $|V(G)| = k$ e $|E(G)| = m$. Per costruire tutte le formule ϕ_v sono necessarie $O(k)$ operazioni, mentre per costruire tutte le formule $\phi_{(v,u)}$ sono necessarie $8O(m)$ operazioni, dunque F impiega $O(k + m)$ operazioni. Tuttavia, notiamo facilmente che $k, m = O(n)$, dunque concludiamo che f sia una riduzione polinomiale per cui $4COL \leq_m^P SAT$.

□

Problema 4.11: NP-Completezza di $HALF-CLIQUE$

Dato il seguente linguaggio:

$$HALF-CLIQUE = \left\{ \langle G \rangle \mid \begin{array}{l} G = (V_G, E_G) \text{ grafo non diretto} \\ \text{con una clique di } \frac{|V_G|}{2} \text{ nodi} \end{array} \right\}$$

dimostrare che $HALF-CLIQUE \in \text{NP-Complete}$

Suggerimento: ricordare che abbiamo dimostrato che SAT , $3SAT$ e $CLIQUE$ siano NP-Completi

Dimostrazione.

- Sia V la TM definita come:

$V =$ "Data la stringa $\langle \langle G \rangle, c \rangle$ in input:

1. Interpreta $c = \langle v_1, \dots, v_k \rangle$, dove $v_1, \dots, v_k \in V_G$
2. Calcola $\frac{|V_G|}{2} = m$
3. Se $m \neq k$, *rifiuta*
4. Altrimenti, ripeti lo step successivo per ogni coppia di nodi v_i, v_j :
 5. Se $(v_i, v_j) \notin E_G$, *rifiuta*
6. *Accetta*"

- Posto $\frac{|V_G|}{2} = m$, per costruzione di V risulta evidente che:

$$\langle G \rangle \in HALF-CLIQUE \iff \text{Esiste una } m\text{-clique in } G \iff$$

$$\exists c \in \Sigma^*, \langle \langle G \rangle, c \rangle \in L(V)$$

implicando che V sia un verificatore per $HALF-CLIQUE$. Inoltre, poiché V svolge solo operazioni polinomiali, concludiamo che $HALF-CLIQUE \in \text{NP}$

- Successivamente, mostriamo che $3SAT \leq_m^P HALF-CLIQUE$
- Sia quindi $f : \Sigma^* \rightarrow \Sigma^*$ la funzione calcolata dalla seguente TM F definita come:

$F =$ "Data la stringa $\langle \phi \rangle$ in input:

1. Costruisci un grafo G definito come:
 - i. Conta il numero k di clausole *or* in ϕ
 - ii. Per ogni clausola, crea un nodo x_i per ogni letterale della clausola, creando dei doppioni se la variabile compare più volte
 - iii. Organizza i nodi di G in k triple (una per clausola *or*), dove tre nodi appartengono alla stessa tripla se e solo se i loro letterali compaiono all'interno di una stessa clausola *or*

- iv. Crea un arco tra ogni coppia di nodi fatta eccezione di nodi appartenenti alla stessa tripla e nodi rappresentanti letterali opposti tra loro (es: x_i e $\overline{x_i}$)
- v. Crea i nodi $v_1, \dots, v_k$
- vi. Per ogni nodo $v_1, \dots, v_k$, crea un arco verso ogni nodo di G

2. Restituisci in output la stringa $\langle G, k \rangle$

- Prima di tutto, notiamo che nel grafo G costruito da F vi siano $3k$ nodi derivati dalle k triple e k -nodi aggiunti alla fine, per un totale di $4k$ nodi
- Supponiamo che $\langle \phi \rangle \in 3SAT$. Poiché ϕ è una 3CNF, affinché essa sia soddisfacibile ne segue che ogni clausola sia soddisfacibile, dunque che almeno uno dei letterali di ognuna di tali clausole sia vero
- Sia quindi $C = \{x_1, \dots, x_k\}$ l'insieme dei nodi tali che $\forall i \in [1, k] \quad x_i$ è il nodo corrispondente al letterale vero della i -esima tripla
- Poiché tali nodi appartengono tutti a triple diverse e poiché $\nexists x_i, x_j \in C$ tali che $x_i = \overline{x_j}$ in quanto non possono essere entrambi veri, ne segue che essi siano tutti due a due adiacenti
- Inoltre, poiché i nodi $v_1, \dots, v_k$ sono adiacenti ad ogni nodo del grafo, concludiamo che $C \cup \{v_1, \dots, v_k\}$ siano una $2k$ -clique (ricordiamo che i nodi totali siano $4k$)
- Di conseguenza, abbiamo che:

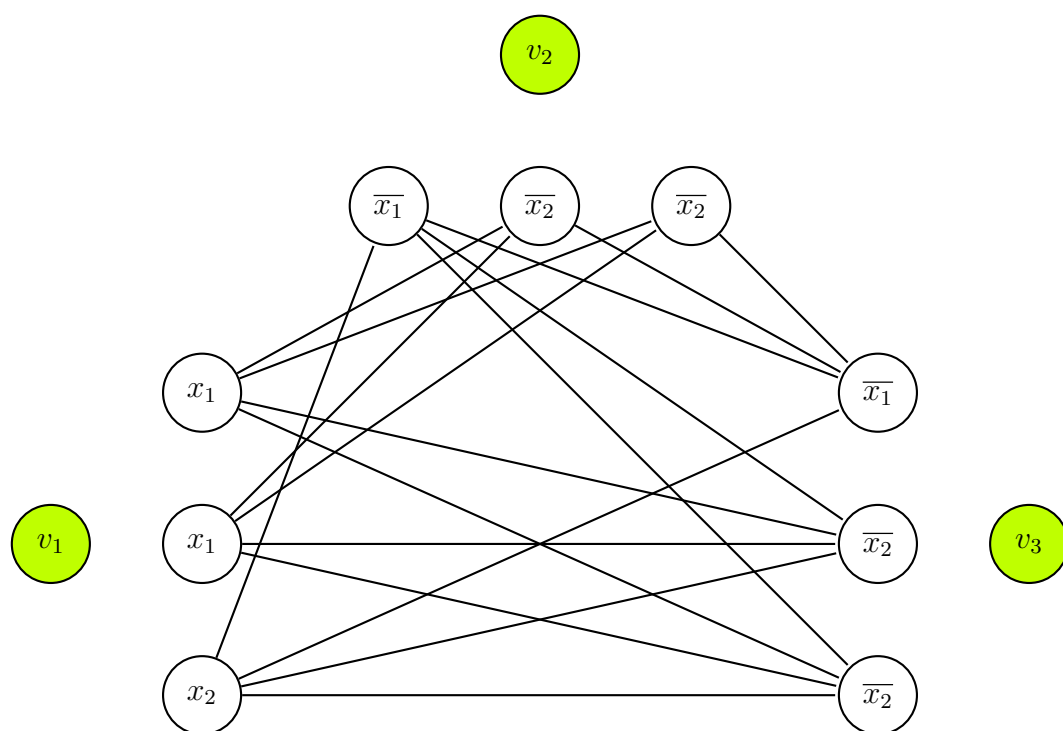
$$\langle \phi \rangle \in 3SAT \implies f(\langle \phi \rangle) = \langle G \rangle \in HALF-CLIQUE$$

- Supponiamo ora che $f(\phi) = \langle G \rangle \in HALF-CLIQUE$, implicando che esista una $2k$ -clique al suo interno. In particolare, k di tali $2k$ nodi corrisponderanno ai nodi $v_1, \dots, v_k$ aggiunti alla fine. Per quanto riguarda gli altri k nodi, per costruzione di G , non esistono nodi all'interno della clique i cui letterali rappresentati appartengono alla stessa tripla
- A questo punto, poiché tali letterali non interferiscono tra loro trovandosi in triple diverse, esiste un un assegnamento che soddisfi ogni clausola. In particolare, tale assegnamento risulta sempre possibile in quanto all'interno della clique non possano appartenere sia x_i che $\overline{x_i}$
- Di conseguenza, abbiamo che:

$$f(\langle \phi \rangle) = \langle G, w \rangle \in HALF-CLIQUE \iff \langle \phi \rangle \in 3SAT$$

Inoltre, poiché F svolge solo operazioni eseguibili in tempo polinomiale, concludiamo che $3SAT \leq_m^P HALF-CLIQUE$ e dunque che $HALF-CLIQUE \in \text{NP-Complete}$

□



Grafo generato dalla funzione f della dimostrazione a partire dalla formula

$$\phi = (x_1 \vee \bar{x}_1 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee x_2 \vee x_2)$$

 I nodi verdi sono adiacenti ad ogni nodo del grafo (archi omessi per chiarezza)

Problema 4.12: NP-Completezza di $DOM-SET$

Dato un grafo G , un *dominant set* di G è un sottoinsieme di vertici $D \subseteq V(G)$ per cui ogni vertice $v \in V(G)$ appartiene a D oppure è adiacente ad vertice in D .

Dato il seguente linguaggio:

$$DOM-SET = \left\{ \langle G, k \rangle \mid \begin{array}{l} G = (V_G, E_G) \text{ grafo non diretto con} \\ \text{un dominant set di massimo } k \text{ nodi} \end{array} \right\}$$

dimostrare che $DOM-SET \in \text{NP-Complete}$

Suggerimento: ricordare che abbiamo dimostrato che SAT , $3SAT$ e $CLIQUE$ siano NP-Completi

Dimostrazione.

- Dato un vertice $v \in V_G$, definiamo $\delta(v)$ come l'insieme composto da v e dai nodi adiacenti a v , ossia $\delta(v) = \{u \in V_G \mid u = v \vee \exists (u, v) \in E(G)\}$
- Sia V la TM definita come:

$V =$ "Data la stringa $\langle \langle G, k \rangle, c \rangle$ in input:

1. Interpreta $c = \langle v_1, \dots, v_h \rangle$ dove $v_1, \dots, v_h \in V_G$
2. Verifica che $h \leq k$. Se falso, *rifiuta*.
3. Per ogni vertice $v \in V_G$, verifica se $\delta(v) \cap \{v_1, \dots, v_h\} \neq \emptyset$. Se falso, *rifiuta*.
4. *Accetta*."

Per definizione stessa di V risulta evidente che $\exists c \in \Sigma^*$ tale che $\langle \langle G, k \rangle, c \rangle \in L(V)$ se e solo se $\langle G, k \rangle \in DOM-SET$. Inoltre, le prime due operazioni di V possono essere svolte in tempo $O(n)$, mentre la terza operazione può essere svolta in tempo $O(n^3)$, concludendo che V sia un verificatore polinomiale per $DOM-SET$ e dunque che $DOM-SET \in \text{NP}$.

- Successivamente, mostriamo che $3SAT \leq_m^P DOM-SET$.
- Sia quindi $f : \Sigma^* \rightarrow \Sigma^*$ la funzione calcolata dalla seguente TM F definita come:

$F =$ "Data la stringa $\langle \phi \rangle$ in input:

1. Conta il numero di variabili m definite in ϕ
2. Costruisci il grafo G definito come:
 - i. Per ogni clausola C_k crea un nodo v_{C_k}
 - ii. Per ogni variabile x definita su ϕ crea i nodi $v_x, v_{\bar{x}}, v_{t_x}$, dove v_{t_x} è un terzo nodo ausiliario, e crea gli archi $(v_x, v_{\bar{x}}), (v_x, v_{t_x})$ e $(v_{\bar{x}}, v_{t_x})$.
 - iii. Per ogni letterale $\ell_{i,k}$ presente nella clausola C_k crea l'arco $(v_{C_k}, v_{\ell_{i,k}})$
3. Restituisci in output la stringa $\langle G, m \rangle$ "

- Supponiamo che ϕ sia soddisfacibile da un assegnamento α . Consideriamo quindi il sottoinsieme $D \subseteq V(G)$ dove per ogni letterale $\ell_{i,k}$ si ha che:

$$v_{\ell_{i,k}} \in D \iff \alpha(\ell_{i,k}) = \text{True}$$

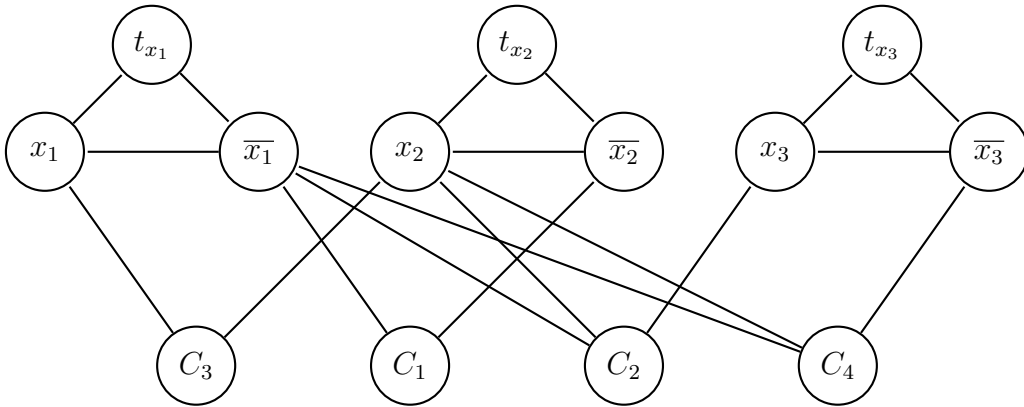
- Poiché in ogni assegnamento, α incluso dunque, un letterale è vero se e solo se il suo letterale negato è falso, ne segue che almeno uno solo tra essi sia presente in D . Dunque, quello non presente in D sarà adiacente a quello presente. Inoltre, poiché α soddisfa ϕ , ogni clausola C_k in ϕ avrà almeno un letterale al suo interno impostato su vero, implicando che C_k sia sempre adiacente ad un vertice in D . Dunque, D è un dominant set di massimo m nodi.
- Viceversa, supponiamo che D' sia un dominant set di massimo m nodi in G . Allora, ogni nodo in D dovrà necessariamente essere un letterale, poiché altrimenti uno dei nodi ausiliari o il nodo di una clausola rimarrebbero scoperti. Inoltre, poiché D ha massimo m nodi, per ogni variabile x solo uno tra i nodi $v_x, v_{\bar{x}}$ sarà presente in D in quanto il numero di variabili definite su ϕ sia m . Consideriamo quindi l'assegnamento α' dove:

$$v_{\ell_{i,k}} \in D' \iff \alpha'(\ell_{i,k}) = \text{True}$$

Dalla costruzione stessa di G risulta evidente che α' soddisfi ϕ .

- Di conseguenza, abbiamo che $\langle \phi \rangle \in 3SAT \iff f(\langle \phi \rangle) = \langle G, m \rangle \in DOM-SET$. Inoltre, poiché F svolge solo operazioni eseguibili in tempo polinomiale, concludiamo che $3SAT \leq_m^P DOM-SET$ e dunque che $DOM-SET \in \text{NP-Complete}$

□



Grafo generato dalla funzione f della dimostrazione a partire dalla formula
 $\phi = (x_1 \vee x_1 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee x_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee \bar{x}_3)$

Problema 4.13: NP-Completezza di *BARRILETE*

Dato un grafo G , un *barrilete cosmico* (o *aquilone cosmico*) di dimensione k è un sottografo $B \subseteq G$ composto da una k -clique con un cammino di k nodi connesso ad un solo nodo della clique.

Dato il seguente linguaggio:

$$BARRILETE = \left\{ \langle G, k \rangle \mid \begin{array}{l} G = (V_G, E_G) \text{ grafo non diretto con} \\ \text{un barrilete cosmico di dimensione } k \end{array} \right\}$$

dimostrare che $BARRILETE \in \text{NP-Complete}$

Suggerimento: ricordare che abbiamo dimostrato che *SAT*, *3SAT* e *CLIQUE* siano NP-Completi

Dimostrazione.

- Sia V la NTM definita come:

$V =$ "Data la stringa $\langle \langle G, k \rangle, c \rangle$ in input, dove $G = (V_G, E_G)$ è un grafo:

1. Interpreta $c = \langle c_1, \dots, c_h, p_1, \dots, p_h \rangle$, dove $c_1, \dots, c_h, p_1, \dots, p_h \in V_G$.
2. Verifica che $h = k$. Se falso, *rifiuta*.
3. Verifica che $c_k = p_1$. Se falso, *rifiuta*.
4. Per ogni coppia c_i, c_j con $i \neq j$, verifica se $(c_i, c_j) \in E(G)$. Se falso, *rifiuta*.
5. Per ogni indice $i = 1, \dots, h - 1$, verifica se $(p_i, p_{i+1}) \in E(G)$. Se falso, *rifiuta*.
6. *Accetta*."

Intuitivamente, V è un verificatore polinomiale per *BARRILETE*, dunque otteniamo che $BARRILETE \in \text{NP}$.

- Successivamente, mostriamo che $CLIQUE \leq_m^P BARRILETE$.
- Sia quindi $f : \Sigma^* \rightarrow \Sigma^*$ la funzione calcolata dalla seguente TM F definita come:

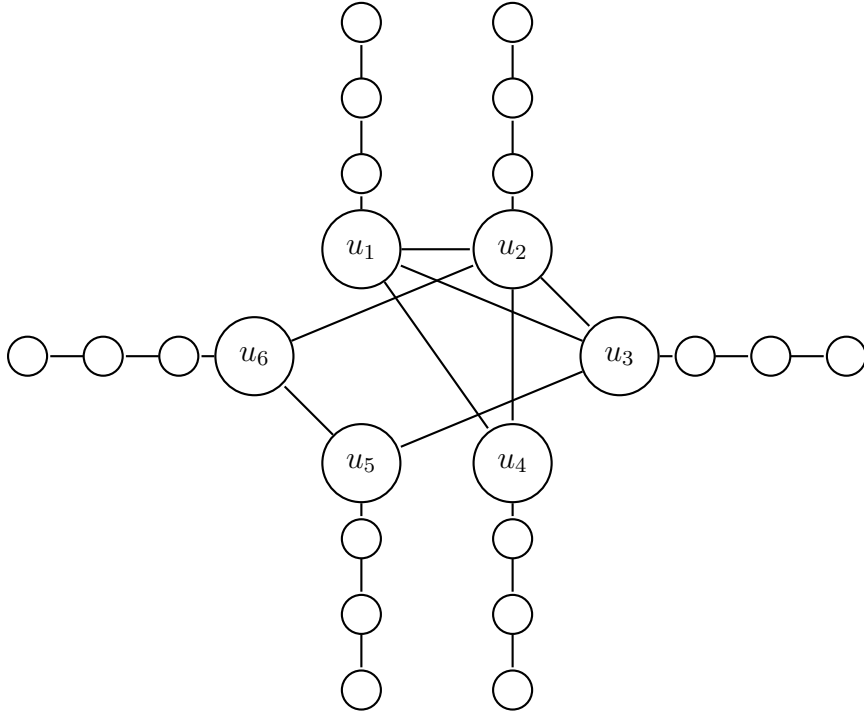
$F =$ "Data la stringa $\langle G, k \rangle$ in input:

1. Costruisci il grafo G' definito come:
 - i. Copia G .
 - ii. Per ogni nodo $u \in V_G$, crea i nodi $v_2, \dots, v_k$ e crea gli archi $(u, v_2), (v_2, v_3), \dots, (v_{k-1}, v_k)$.
2. Restituisci in output la stringa $\langle G', k \rangle$ "

- Supponiamo quindi che $\langle G, k \rangle \in CLIQUE$. In tal caso, esiste una k -clique C in G , dunque tramite un qualsiasi cammino $P = u, v_2, \dots, v_k$ aggiunto in G' otteniamo che $C \cup P$ sia un barrilete cosmico di dimensione k in G' , dunque $\langle G', k \rangle \in BARRILETE$.

- Viceversa, se $\langle G', k \rangle \in \text{BARRILETE}$ contiene un barrilete cosmico B di dimensione k allora per costruzione stessa di G' la k -clique in B deve per forza essere anche in G , dunque $\langle G, k \rangle \in \text{CLIQUE}$.
- Di conseguenza, abbiamo che $\langle G, k \rangle \in \text{CLIQUE} \iff f(\langle G, k \rangle) = \langle G', k \rangle \in \text{BARRILETE}$. Inoltre, poiché F svolge solo operazioni eseguibili in tempo polinomiale, concludiamo che $\text{CLIQUE} \leq_m^P \text{BARRILETE}$ e dunque che $\text{BARRILETE} \in \text{NP-Complete}$

□



*Grafo generato dalla funzione f della dimostrazione.
I nodi $u_1, u_2, u_3, u_4, u_5, u_6$ sono i nodi originali del grafo.*

Problema 4.14

Si considerino i seguenti due linguaggi:

$$\text{FACTOR} = \{ \langle m, k \rangle \mid \exists p \in \mathbb{P} \text{ tale che } p \mid m \text{ e } p \leq k \}$$

$$\text{PRIMES} = \{ \langle n \rangle \mid n \in \mathbb{P} \}$$

Assumendo che $\text{PRIMES} \in P$, dimostrare che se FACTOR è NP-Completo allora $\text{NP} = \text{coNP}$

Soluzione:

- Per dimostrare il risultato richiesto, è sufficiente dimostrare che $\text{FACTOR} \in \text{coNP}$. Per ottenere ciò, è sufficiente definire un verificatore di tempo polinomiale per il linguaggio $\overline{\text{FACTOR}}$.

- Sia M la TM che decide il linguaggio $\text{PRIMES} = \{\langle n \rangle \mid n \text{ is prime}\}$. Per assunzione del problema, sappiamo che $\text{PRIMES} \in \text{P}$ (fun fact: questo risultato è in realtà vero, ma non viene coperto nel corso).
- Definiamo prima il nostro verificatore V :
 $V = \text{"Su input } \langle \langle m, k \rangle, c \rangle$:
 1. Verifica che $\langle c \rangle = \langle p_1, \dots, p_t \rangle$ dove $p_1, \dots, p_t$ sono numero interi non-negativi. Se falso, rifiuta.
 2. Per ogni $i = 1, \dots, t$:
 3. Verifica che $M(p_i)$, ossia che p_i sia primo (in tempo polinomiale). Se falso, V rifiuta.
 4. Verifica che p_i divida m . Se falso, V rifiuta.
 5. Verifica che $p_i \leq k$ per ogni $i = 1, \dots, t$. Se falso, V rifiuta.
 6. Verifica che $m = p_1 \cdot \dots \cdot p_t$. Se falso, V rifiuta.
 7. Se tutti i controlli vengono superati, V accetta"
- Intuitivamente, questo verificatore controlla che il certificato in input sia la fattorizzazione di m , assicurandosi che nessun fattore sia minore di k .
- Tuttavia, è importante osservare un fatto cruciale: dobbiamo garantire che il numero t di fattori sia polinomiale rispetto alla lunghezza dell'input $\langle m, k \rangle$, altrimenti non sarà possibile leggere l'intero certificato in tempo polinomiale.
- Osserviamo che il caso peggiore, ossia che m abbia il massimo numero di fattori possibili, si verifica quando $m = 2^t$. Per tanto, abbiamo che $t = \log m$, confermando che t sia polinomiale rispetto alla lunghezza dell'input, il quale è lungo $O(\log m + \log k)$ bit.
- Per tanto, concludiamo che $\overline{\text{FACTOR}} \in \text{NP}$ in quanto tutte le operazioni svolte da V richiedono tempo polinomiale.

Problema 4.15

Sia $L = \{w \in \{a, b\}^* \mid |w|_a = 2|w|_b\}$. Si descriva una TM a singolo nastro M_1 che decide L in tempo $O(n^2)$. Inoltre, si descriva una TM multinastro M_2 che decide L in tempo $O(n)$.

Soluzione:

- Definiamo la seguente TM M_1 per la prima richiesta del primo punto:

$M_1 = \text{"Su input } w$:

1. Muovi la testina sulla cella più a sinistra
2. Ripeti se la testina non si trova sul carattere speciale $\sqcup$:

3. Se il carattere sotto la testina è $'a'$, sostituiscilo con il carattere $'x'$ e continua a scorrere a destra finché non vengono trovati due caratteri $'b'$, sostituendoli con il carattere $'y'$. Se i due caratteri vengono trovati, sposta la testina a sinistra fino al primo carattere $'x'$ letto. Altrimenti, rifiuta se viene trovato il carattere $\sqcup$.
4. Se il carattere sotto la testina è $'b'$, sostituiscilo con il carattere $'x'$ e continua a scorrere a destra finché non vengono trovati un carattere $'b'$ e un carattere $'a'$, con il carattere $'y'$. Se i due caratteri vengono trovati, sposta la testina a sinistra fino al primo carattere $'x'$ letto. Altrimenti, rifiuta se viene trovato il carattere $\sqcup$.

5. Accetta"

- È facile vedere che M_1 decida L in tempo $O(n^2)$ in quanto nel caso peggiore ogni simbolo dell'inout richiederà di scorrere tutto l'input
- Per il secondo punto, invece, possiamo sfruttare la presenza di più nastri per contare il numero di $'a'$ e il numero di $'b'$ per poi confrontarli.
- Tuttavia, osserviamo come l'aggiornamento di due contatori binari abbia costo $O(\log n)$, dunque ripetere tale operazione per potenzialmente n caratteri avrebbe costo totale $O(n \log n)$, sfidando il bound richiesto.
- Per risolvere tale problema, utilizziamo due contatori unari. Ad esempio, ogni volta che leggiamo un $'a'$ aggiungiamo un simbolino 1 al contatore unario. A differenza del contatore binario, tale aggiornamento ha costo $O(1)$, rendendo il costo totale $O(n)$.
- Consideriamo quindi la seguente macchina a 3 nastri M_2 .

$M_2 =$ "Su input w :

1. Muovi la testina sulla cella più a sinistra
2. Ripeti se la testina del nastro 1 (quello di input) non si trova sul carattere speciale $\sqcup$:
 3. Se il carattere sotto la testina del nastro 1 è $'a'$, scrivi $'1'$ sotto la testina del nastro 2 e sposta entrambe le testine a destra di una cella
 4. Se il carattere sotto la testina del nastro 1 è $'b'$, scrivi $'1'$ sotto la testina del nastro 3 e sposta entrambe le testine a destra di una cella
5. Sposta le testine del nastro 2 e 3 sulle loro prime celle
6. Ripeti se la testina del nastro 2 non si trova sul carattere speciale $\sqcup$:
 7. Sposta la testina del nastro 2 di una cella a destra
 8. Sposta la testina del nastro 3 di una cella a destra
 9. Se entrambe le testine si trovano sul carattere speciale $\sqcup$, accetta
10. Rifiuta"